



MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

A Bottom-Up Layered Queueing Network Solver: Construction and Characterisation

Author:
Alex Slater

Supervisor:
Dr. Giuliano Casale

Second Marker:
Dr. Ahmed Fetit

12th June 2026

Abstract

Layered queueing networks (LQNs) extend classical queueing networks by modelling synchronous calls between layered services and the blocking they induce, enabling the analysis of distributed software systems. LQNS and LINE’s **SolverLN** are two general-purpose tools for solving LQN models, however they have become bloated over years of development. This report describes the construction of **SolverLNSimple**, a simplified LQN solver, built as the minimal solution under the paradigm of only adding necessary features based on the set of models considered. The build proceeds through a series of features, each triggered by a specific case of LQN model the existing design failed to handle: ensemble prefix coupling, graph-derived coupling, multi-class models, an MVA-kernel overhaul, larger fan-out callers and activity graph integrations. This bottom-up discipline restricts the machinery added, providing a lightweight, efficient solver for a large class of LQN models. Once the solver was sufficiently correct, its non-generality was exploited to discover performance benefits. An evaluation of the solver, on a 78-fixture corpus, against **SolverLN** and LQNS verifies its correctness and characterises its performance, where we see a large performance win whilst maintaining accuracy on the set of models the solver claims to handle. 73.6% of the iteration count difference is attributable to **SolverLN**’s `iter_min` convergence floor, with the rest reflecting genuine algorithmic difference, however the omission of a moving-average convergence detection and interlock machinery carries a small, quantified accuracy cost. Regarding runtime, much of the difference is attributed to **SolverLN**’s interlock correction, where cost grows with fan-out breadth, and a computed-but-not-consumed task-caller traversal.

Acknowledgements

I would like to thank my supervisor, Dr. Giuliano Casale, for his guidance throughout this project. His maintenance of the LINE codebase, specifically **SolverLN**, provided the primary comparator for my solver, and his time and insights have been invaluable.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	2
2	Background	3
2.1	Queueing Network Modelling	3
2.2	Mean Value Analysis	7
2.3	Extended Queueing Networks	12
2.4	Solving Layered Queueing Networks	14
3	Contribution	21
3.1	Coupling on the ensemble	21
3.2	Graph-derived coupling	23
3.3	Multi-class support	25
3.4	An MVA kernel for LQN-shaped inputs	28
3.5	Fan-out callers	29
3.6	Activity-graph awareness	31
3.7	Optimising the solver	34
3.8	Closing remarks	38
4	Evaluation	39
4.1	Scope and Hypotheses	39
4.2	Instrumentation	40
4.3	Correctness	41
4.4	Iteration-count comparison	42
4.5	Runtime comparison	44
4.6	SolverLN ablations	46
4.7	The accuracy boundary	50
4.8	Findings	51
5	Conclusions	54
5.1	Assessing the result	54
5.2	Is SolverLNSimple a defensible architecture?	55
5.3	The surfaced algorithmic candidates	55
5.4	Out of scope vs Out of reach	56
5.5	Field-level reflection	56
5.6	Future work	57
5.7	Conclusion	57
	Declarations	61
	Appendix	62

List of Figures

2.1	A computer system model.	4
2.2	Application of Little’s Law.	6
2.3	Precedence of exact multi-class MVA intermediate solutions.	11
2.4	The four-step LQN solver pipeline	15
2.5	Outer fixed-point loop wrapping per-submodel inner MVA solves	16
2.6	Two-submodel ensemble decomposition and coupling for the worked example	17
3.1	The three blind spots motivating graph-derived coupling.	23
3.2	The multi-class fixture categories.	25
3.3	Motivation for AMVA dispatch and primitive <code>MvaLd/MvaAmva</code>	30
3.4	An example activity graph model.	31
3.5	Top CPU samples in the pre-optimisation JFR profile.	35
4.1	Per-fixture max relative difference on Partition A	42
4.2	Outer-loop iteration counts per solver, faceted by partition	43
4.3	Per-fixture wall-clock runtime per solver, fair-bracket Sim, medianed	45
4.4	Per-fixture runtime, LQNS net of REST overhead	46
4.5	Interlock-off per-iteration speedup on Partition B	48
4.6	Per-partition iteration-count gap closure	50

Chapter 1

Introduction

1.1 Motivation

Modern software systems have become increasingly more complex due to the widespread adoption of distributed, multi-tier architectures. Client-server and microservice-based systems [1] are commonly used to achieve scalability and flexibility; however, they also introduce significant performance analysis challenges [2]. Synchronous request-reply interactions, limited server concurrency and shared hardware resources can lead to blocking, contention and non-linear behaviour [3], making it difficult to predict system performance.

A well-established approach to understanding such systems is an analytical performance modelling technique known as queueing network models. A queueing network models a system as a collection of interconnected service stations, each with an associated queue of jobs awaiting service. By analysing these models, it is possible to derive important performance metrics such as average response time, throughput, queue length and resource utilisation [4]. These can be used to guide architectural decisions and identify bottlenecks early in the design process of computer systems.

Queueing network models have been widely applied within computer systems research, including the analysis of processors and I/O devices, transaction processing systems and distributed applications [4]. Their importance has increased as modern systems demand high throughput and low latency, allowing performance considerations to be addressed before implementation. Compared to techniques such as measurement or simulation, analytical models allow rapid exploration of system designs at a relatively low cost, making them particularly valuable during the initial stages of development [5].

In this project, we focus on the application of queueing models on distributed server systems. Whilst analytical techniques on classical queueing network models allow for efficient solutions, they rely on the assumption of product-form behaviour and a lack of blocking [4], which are frequently violated in client-server systems. For example, synchronous remote procedure calls introduce simultaneous resource possession, where both client and server resources are held during a call, invalidating the product-form independence assumptions [5].

To address these limitations, Layered Queueing Networks (LQNs) were introduced as a natural extension: able to represent software entities, such as tasks and entries, with their limited concurrency and synchronous interactions, while also modelling hardware resources, such as processors. By structuring the model into layers and representing synchronous calls as rendezvous interactions, LQNs are able to capture blocking, nested service demands and contention effects that arise naturally in multi-tier systems [5, 6, 7].

However, the increased modelling power of LQNs comes at the cost of increased computational and implementation complexity. LQN solvers rely on decomposition techniques, surrogate delays and approximate mean-value analysis (MVA) variants, combined with fixed-point iteration [4, 5]. These techniques have been used in LQNS and other advanced solver tools, which support a wide range of modelling constructs such as interlocking, multiservers and complex

activity graphs [5]. While this generality enables the analysis of realistic systems, it also results in large, complex codebases that are difficult to understand, modify, or extend and overall lead to less efficient solvers.

This project aims to explore the trade-off between modelling power and solver complexity by implementing a simplified LQN solver inspired by LQNS. Rather than reproducing the full functionality of existing solvers, the project focuses on a restricted but representative subset of LQN features, extending the solver only when necessary. The objective is to develop a solver that is easier to understand and extend, solving LQNs more efficiently, while retaining a high level of accuracy. This will in turn allow us to explore the design of existing LQN solvers, identifying the key pieces of machinery that contribute to their performance and accuracy.

1.2 Contributions

The report makes the following contributions:

1. **SolverLNSimple, a simplified LQN solver built bottom-up** Construction of this solver began from a minimal baseline solution, with six sets of features added, each driven by a specific class of LQN model the existing design could not handle. Initially delegating the inner MVA kernel to the same closed-form routine the established comparators use, we focused on writing the outer coupling iteration from scratch. The solver handles single-class chains, fan-out caller topologies, multi-class shared-callee topologies, multi-entry and reference-task models, multi-server processors, deep infinite-server topologies, and the activity-graph feature set. A second phase then optimises the iteration count and runtime of the solver, exploiting its non-generalised design in a way general-purpose solvers cannot. The solver matches **SolverLN/LQNS** within standard tolerance on closed LQN models, with an improvement in iteration count and execution time.
2. **An evaluation of SolverLNSimple against SolverLN and LQNS** On a 78-fixture evaluation corpus developed to consider a wide range of model classes, the simple solver matches either **SolverLN**'s or **LQNS**'s outputs within standard tolerance on all fixtures we expect it to, whilst outperforming both on iteration count and execution time. An ablation study of **SolverLN** is also performed, isolating its `iter_min` floor and moving-average convergence detection, as well as its unconditional interlock traversal and detection logic, enabling us to quantify each machinery's contribution to the gap in iteration count to **SolverLNSimple**. The combined decomposition accounts for 73.6% of the corpus-aggregate iteration-count difference, leaving a 26.4% residual identified structurally as genuine algorithmic difference. We also explore the small accuracy cost occurring as a result of these omissions.

Chapter 2

Background

This chapter provides the queueing network vocabulary the rest of the report relies on. We first introduce the modelling structures and laws in Section 2.1, look at how these laws can be manipulated to solve queueing network models through Mean Value Analysis (MVA) in Section 2.2, and then consider the modelling situations in which the assumptions break and look at the client-server extensions that motivate Layered Queueing Networks in Section 2.3.

2.1 Queueing Network Modelling

Queueing networks provide a formal abstraction for evaluating performance in computer systems, where jobs (or *customers*) contend for shared system resources. These networks enable analysts to derive key metrics, such as utilisation, throughput and response time, without requiring fully built systems and intense simulation. This section introduces the structure and principles of underlying queueing models, including their components, performance laws and outputs, with a focus on separable product-form models.

2.1.1 Basics

A queueing network model represents a system using two primary entities: *customers*, which model users or transactions, and *service centres*, which model resources such as CPUs, disks, or servers. These components have a close correspondence to the attributes of real computer systems, allowing them to be modelled naturally.

Customers

The behaviour of customers can be characterised into three types, enabling the modelling of systems of various forms, as specified by the *workload intensities* shown below [4]:

- A **transaction workload** represents independent requests arriving from the outside world, behaviour typical of API calls and database queries. This arrival-driven workload is specified by the arrival rate λ at which jobs arrive into the system. Customers are serviced and then leave the model on completion, causing a population that varies over time.
- A **batch workload** represents a fixed number of jobs inside the system. This is typical of analytical jobs, and it works under the assumption that when a job finishes and leaves the system it is immediately replaced by another job from the same workload. This ensures a constant population size, specified by parameter N , relating to the average population size of the true workload.
- A **terminal workload** models jobs that pause between requests, typical of timesharing systems and interactive users. Again, the population is fixed by parameter N , however instead of exiting the system and being replaced immediately, the user *thinks* for a specified time Z before submitting the next request.

A model that contains only transaction workloads is known as an **open model**, as there is an infinite stream of incoming transactions causing varied population size, whilst a model that contains only batch and terminal workloads is a **closed model** as it consists of a fixed number of customers that re-circulate. A model that consists of both open and closed classes is known as a mixed model [4].

For the purpose of this project, we will only consider closed models. This is a deliberate scoping decision as closed models are the standard formalism for enterprise software performance analysis since interactive users, microservice request loops and bounded job pools all naturally circulate a fixed customer population through the system rather than admitting an unbounded external stream [4].

Service Centres

Service centres represent any system resource that provides service to customers, and can be of two different types [4]:

- **Queueing centres** are service centres where customers compete for a limited resource. Customers may experience a *wait time* whilst they are in the queue before the resource becomes available, they then spend the necessary *service time* at the resource. Queueing centres are used to model computing resources such as CPU or disk. A scheduling discipline (FCFS, LCFS, SIRO, PS, IS) can also be established for models with multiple types of customers [8, 9].
- **Delay centres** are service centres where customers do not compete, instead treated as having their own server. This means there is no queue and therefore no wait time, just the service time specified. Delay centres can be used to model user think time or fixed communication delays.

Service Demands

Service centres must specify the amount of time required for a customer to be served, denoted D_k for service centre k . The service demand can be thought of as B_k/C , where B_k is the total busy time at service centre k and C is the total number of system completions, or as $V_k \cdot S_k$, where V_k is the number of visits made to service centre k and S_k is the service requirement per visit to k . We let D be the total service demand for customers at all centres, such that $D \equiv \sum_{k=1}^K D_k$ [4].

Visualising Models

Now that we have an understanding of these models, we are able to visualise them. Figure 2.1 shows how these service centres can be combined and applied to a real computing problem. This model characterises a computer system, representing each system resource (a CPU and three disks) as separate queueing centres and representing customer arrival via a delay centre [4].

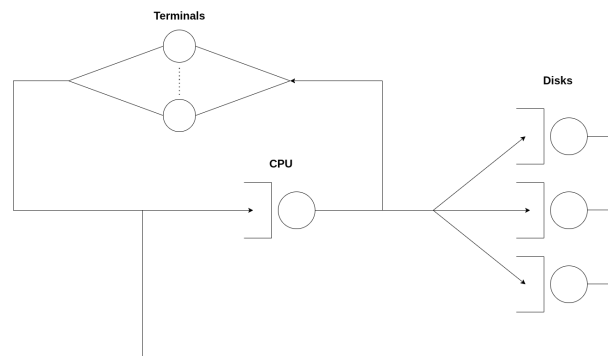


Figure 2.1: A computer system model.

2.1.2 Laws

It is now possible to look into some of the fundamental laws of queueing network systems that will help form our analysis techniques.

Basic Quantities

It is first worth considering the system as a whole. We can measure the **observation period** T , the number of **arrivals** A , the number of **completions** C , and the **busy time** B for which the resource was observed to be in use. From these we can calculate the **arrival rate** $\lambda = A/T$, the **throughput** $X = C/T$, the **utilisation** $U = B/T$, and **mean service requirement** $S = B/C$ of jobs [4].

Utilisation Law

By considering that $B/T = (C/T)(B/C)$, we can derive the utilisation law, stating that the utilisation of the system is equal to the product of the throughput and the service requirement [4]:

$$U = X \cdot S \quad (2.1)$$

Little's Law

The most useful law we will come across is Little's Law. First we need to define a few new quantities: the **accumulated time in the system** W , the **average number of requests in the system** $N = W/T$, and the **average system residence time per request** $R = W/C$ [4].

We can now derive Little's Law from the fact that $W/T = (C/T)(W/C)$. This law states that the average population of the system is equal to the product of the throughput of the system and the average time a request spends in the system [4]:

$$N = X \cdot R \quad (2.2)$$

This law is important as it is widely applicable, requires only weak assumptions, and is valuable in checking the consistency of data measurements. It is also frequent for us to know two of the three quantities of the equation when studying computer systems, allowing us to calculate the third. Finally, Little's Law is the core part of the algorithms we discuss later on for evaluating queueing network models [4].

Little's Law application

Figure 2.2 revisits the queueing network we established earlier and lets us apply Little's Law at four levels of the system:

Level 1: a single resource (server only). Population is utilisation (0 or 1), throughput is the rate at which the resource satisfies requests, and residence time is the mean service requirement per request.

Level 2: the resource with its queue. Population is the total number of requests in the queue or server, throughput is unchanged, and residence time is the mean time a request spends queueing and being served.

Level 3: the central subsystem. Population is the number of customers in the central system, throughput is the rate at which interactions flow between terminals and the system, and residence time is the time between a user request and response.

Level 4: the full system, including terminals. Population is the total number of users, throughput is unchanged, and residence time is the sum of system response time and user think time.

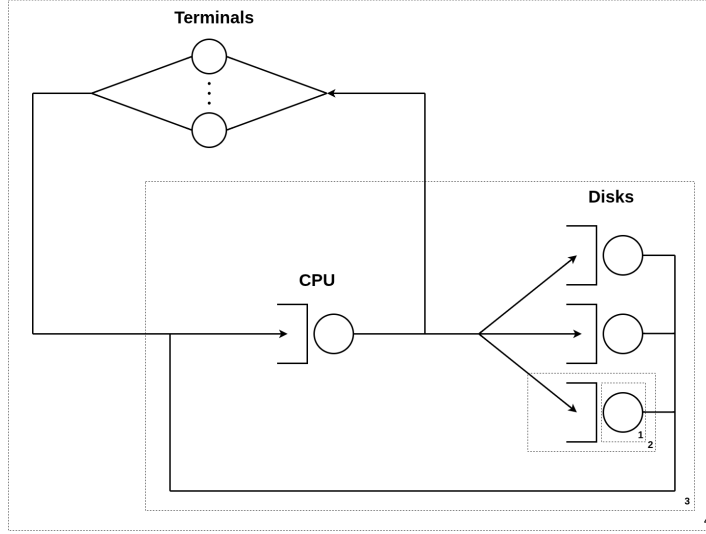


Figure 2.2: Application of Little's Law.

The Response Time Law

Treating the system as a whole lets us split residence time into response time and think time, giving $N = X(R + Z)$, from which we can derive the response time law [4]:

$$R = \frac{N}{X} - Z \quad (2.3)$$

The Forced Flow Law

Another fundamental law of queueing networks states that throughput in all parts of the system must be proportional. This becomes an extremely powerful tool when combined with Little's Law. To define this we must first define the **visit count** of resource k as $V_k = C_k/C$. Rearranging this as $C_k = V_k \cdot C$ and dividing through by T allows us to use the definition of throughput $X = C/T$ to give us the forced flow law [4]:

$$X_k = V_k \cdot X \quad (2.4)$$

The Flow Balance Assumption

It may be useful to assume that the closed systems we are looking at satisfy the assumption that the number of arrivals is equal to the number of completions, $A = C$. Equivalently, the arrival rate to the system equals the throughput of the system, $\lambda = X$. This can be used in conjunction with Little's Law and the Forced Flow Law to calculate device utilisations for systems with terminal workload intensities [4].

2.1.3 Outputs

Once a queueing model is defined and evaluated, we are able to obtain a list of outputs useful for performance analysis. As the inputs to the model are described as averages over time, so will the outputs. We can show that the value of an output relates to a specific input through notation such as $X(N)$, where X is the throughput for a batch or terminal class with population N . We now discuss key performance measures that will help us analyse our system [4].

Utilisation

Denoted U_k , the utilisation of centre k is defined as the proportion of time device k is busy or, for a delay centre, the average number of customers in service there. We can use the utilisation law $U_k = X_k \cdot S_k$ to calculate this for each device [4].

Residence Time

The total residence time of a customer at centre k , denoted R_k , describes the time a customer spends at the centre over all visits in a request to the system. This allows us to find R , the system response time: $R = \sum_k R_k$, which finds how long a customer stays in the system by summing all centres' residence times [4].

Throughput

The throughput at centre k , denoted X_k , is the rate of job completions. If the system is parametrised in terms of V_k and S_k , we can find the throughput of each centre with $X_k = V_k \cdot X$. If it is parametrised with D_k directly, we can only determine the throughput of the entire system, X [4].

Queue Length

The average queue length at centre k , denoted Q_k , includes both customers being served and customers waiting to be served. Therefore, the number of customers waiting can be calculated as $Q_k - U_k$. We let Q denote the average number of customers in the system. For a batch class, $Q = N$ (the fixed population within the subsystem). For a terminal class, $Q = N - X \cdot Z$ (the total population minus those who are thinking) [4].

2.1.4 Multi-Class Models

So far, we have only considered models with a single homogeneous workload class, in which all jobs exhibit identical service demands and routing behaviour. Whilst this abstraction makes it easy to determine and evaluate our system, it may not be accurate enough in practice. Real computer systems typically serve multiple classes of customers, with differing arrival rates and service demands. We must be able to model these more closely.

Inputs

The workload intensity for the entire network can be classified by a vector of intensity parameters $\mathbf{N} = (N_1, N_2, \dots, N_C)$ for a closed network, where each value corresponds to the population of a terminal or batch workload. Each customer class will have its own service demand at each centre, specified by $D_{c,k}$. We do not need to specify the scheduling discipline at queueing centres, as we make the assumption it is class-independent [4].

Outputs

All performance measures for multi-class systems can be obtained on either a per-class or an aggregated basis. For utilisation, queue length and throughput, the aggregated performance measure equals the sum of the per-class performance measures: $U = \sum_{c=1}^C U_c$, $Q = \sum_{c=1}^C Q_c$, $X = \sum_{c=1}^C X_c$. For residence time and system response time, the per-class measures must be weighted by relative throughput to achieve the aggregated measures: $R = \sum_{c=1}^C R_c \cdot X_c / X$, $R_k = \sum_{c=1}^C R_{c,k} \cdot X_c / X$ [4].

2.2 Mean Value Analysis

Having established the structure of queueing network models, their inputs and the fundamental performance laws, we now need to establish our solution technique. Mean Value Analysis (MVA) provides an efficient analytic technique for solving queueing networks by iteratively computing mean performance measures without requiring explicit enumeration of the underlying Markov state space. By exploiting product-form properties and conservation laws, MVA enables the evaluation of systems with multiple service centres and customer classes at a computational cost much less than that of simulation or experimentation. As a result, MVA has become a widely used solution technique for performance analysis of computer systems.

This chapter develops the *foundational* MVA recursion. MVA was introduced by Reiser and Lavenberg [10] for closed multichain multi-class, with the single-class recursion being a special case. We follow the textbook approach from Lazowska et al. [4], representing the single-class

case first and then generalising to multiple classes.

2.2.1 Single-Class MVA

First, we will establish the fundamentals of MVA on single-class models. Mean Value Analysis is based on three equations, derived from the fundamental laws we established earlier. These are [4]:

- Little's Law applied to the queueing network as a whole:

$$X(N) = \frac{N}{Z + \sum_{k=1}^K R_k(N)} \quad (2.5)$$

- Little's Law applied to the service centres individually:

$$Q_k(N) = X(N) \cdot R_k(N) \quad (2.6)$$

- Service centre residence time equations:

$$R_k(N) = \begin{cases} D_k & \text{(delay centres)} \\ D_k \cdot (1 + A_k(N)) & \text{(queueing centres)} \end{cases} \quad (2.7)$$

where $A_k(N)$ represents the average number of customers seen at service centre k on the arrival of a new customer.

Equations (2.5) and (2.6) can be computed if residence times are known, so (2.7) must be calculated first. This can be done with an exact approach, providing a direct solution, or with an approximate solution, providing bounds for the solution. Both techniques are accurate, the choice comes down to the trade-offs between requiring an exact solution and the time and space required to do so.

Exact Single-Class MVA

Closed separable networks hold the property that the *arrival instant queue length* of service centre k for a network population of N is equal to the average queue length at the service centre for a population of $N - 1$. Intuitively, when a customer arrives at the station, we know it is not already in the queue, so we can consider the behaviour of the system without this customer [4]:

$$A_k(N) = Q_k(N - 1) \quad (2.8)$$

Our MVA algorithm works by observing that all queue lengths are zero when there are no customers in the network. We then use $A_k(1) = Q_k(0) = 0$ to get the arrival instant queue length for a network population of one. We continue with this premise, iterating and calculating our metrics at each step until our solution is built up to the population size we require. This process is possible due to the equations (2.5)–(2.8) we defined earlier [4, 10].

Algorithm 2.1: Exact MVA for single-class models [4]

```

1:   $Q_k(N) \leftarrow 0$  for all  $k = 1, \dots, K$ 
2:  for  $n = 1$  to  $N$  do
3:    for  $k = 1$  to  $K$  do
4:       $R_k \leftarrow \begin{cases} D_k & \text{(delay centres)} \\ D_k(1 + Q_k) & \text{(queueing centres)} \end{cases}$ 
5:    end for
6:     $X \leftarrow \frac{n}{Z + \sum_{k=1}^K R_k}$ 
7:    for  $k = 1$  to  $K$  do
```

```

8:       $Q_k \leftarrow X R_k$ 
9:    end for
10: end for

```

On termination of the algorithm, the values immediately available to us are **system throughput** X , device k **queue length** Q_k and device k **residence time** R_k . We are also able to use Little's Law to calculate the **system response time** $N/X - Z$, the **average system population** $N - X \cdot Z$, the device k **throughput** $X \cdot V_k$ and the device k **utilisation** $X \cdot D_k$ [4].

The time complexity of this algorithm is $O(N \cdot K)$ arithmetic operations, as each equation is applied N times, looping over each of the K service centres. The space complexity is $O(K)$, as we do not need to store intermediate results, just the current value of the three required metrics for each service centre [4, 10].

Approximate Single-Class MVA

An approximate version of MVA can be used when we require a more efficient solution. To do this, we replace the exact equation $A_k(N) = Q_k(N - 1)$ with the estimate $A_k(N) \approx h[Q_k(N)]$, where we must give a suitable approximation function h that depends on queue length. One simple and accurate approximation of h is $h[Q_k(N)] \equiv \frac{N-1}{N} \cdot Q_k(N)$, based on the assumption that $Q_k(N)/N = Q_k(N-1)/(N-1)$ for all k [11]. This algorithm works similarly to the exact version, however instead of iterating up to the population size, which depends on all previous population sizes, it can instead iterate around an estimate for the required population directly, until convergence [4].

Algorithm 2.2: Approximate MVA for single-class models [4]

```

1:   $Q_k(N) \leftarrow \frac{N}{K}$  for all  $k = 1, \dots, K$ 
2:  repeat
3:     $Q_k^{\text{old}}(N) \leftarrow Q_k(N)$  for all  $k = 1, \dots, K$ 
4:    for  $k = 1$  to  $K$  do
5:       $A_k(N) \leftarrow h[Q_k(N)]$ 
6:    end for
7:    for  $k = 1$  to  $K$  do
8:       $R_k(N) \leftarrow \begin{cases} D_k & \text{(delay centres)} \\ D_k(1 + A_k(N)) & \text{(queueing centres)} \end{cases}$ 
9:    end for
10:    $X \leftarrow \frac{N}{Z + \sum_{k=1}^K R_k(N)}$ 
11:   for  $k = 1$  to  $K$  do
12:      $Q_k(N) \leftarrow X R_k(N)$ 
13:   end for
14: until  $\max_k |Q_k(N) - Q_k^{\text{old}}(N)| \leq \epsilon$ 

```

2.2.2 Multi-Class MVA

We now move on to the Mean Value Analysis technique for multi-class systems, where we allow modelling of jobs with significantly different behaviours, and can give outputs in terms of individual classes. However, having multiple sets of input parameters does mean gathering the input data becomes more tedious, along with the fact that most current measurement tools are not sufficient. The solution techniques also become more difficult to implement. For multi-class solutions, we adapt the equations (2.5)–(2.7) previously seen [4]:

- For each class, Little's Law applied to the queueing network as a whole:

$$X_c(\mathbf{N}) = \frac{N_c}{Z_c + \sum_{k=1}^K R_{c,k}(\mathbf{N})} \quad (2.9)$$

- For each class, Little's Law applied to the service centres individually:

$$Q_{c,k}(\mathbf{N}) = X_c(\mathbf{N}) \cdot R_{c,k}(\mathbf{N}) \quad (2.10)$$

It may also be useful to consider total queue length at centre k :

$$Q_k(\mathbf{N}) = \sum_{c=1}^C Q_{c,k}(\mathbf{N})$$

- For each class, service centre residence time equations:

$$R_{c,k}(\mathbf{N}) = \begin{cases} D_{c,k} & \text{(delay centres)} \\ D_{c,k} \cdot (1 + A_{c,k}(\mathbf{N})) & \text{(queueing centres)} \end{cases} \quad (2.11)$$

Therefore, similarly to single-class MVA, once $A_{c,k}(\mathbf{N})$ is known, the above can be calculated.

Exact Multi-Class MVA

Like the single-class case, the exact value of the arrival instant queue length is the average queue length at the service centre with one less customer in the population: $A_{c,k}(\mathbf{N}) = Q_k(\mathbf{N} - \mathbf{1}_c)$. We can once again iteratively construct our solution, starting with the knowledge that $Q_k(\mathbf{0}) = 0$, iterating through every possible permutation of populations up to the desired [4, 10].

Algorithm 2.3: Exact MVA for multi-class models [4]

```

1:   $Q_k(\mathbf{0}) \leftarrow 0$  for all  $k = 1, \dots, K$ 
2:  for  $n = 1$  to  $\sum_{c=1}^C N_c$  do
3:    for each feasible population  $\mathbf{n} \equiv (n_1, \dots, n_C)$  with  $n$  total customers do
4:      for  $c = 1$  to  $C$  do
5:        for  $k = 1$  to  $K$  do
6:           $R_{c,k} \leftarrow \begin{cases} D_{c,k} & \text{(delay centres)} \\ D_{c,k}(1 + Q_k(\mathbf{n} - \mathbf{1}_c)) & \text{(queueing centres)} \end{cases}$ 
7:        end for
8:      end for
9:      for  $c = 1$  to  $C$  do
10:        $X_c \leftarrow \frac{n_c}{Z_c + \sum_{k=1}^K R_{c,k}}$ 
11:     end for
12:     for  $k = 1$  to  $K$  do
13:        $Q_k(\mathbf{n}) \leftarrow \sum_{c=1}^C X_c R_{c,k}$ 
14:     end for
15:   end for
16: end for

```

Similarly to the single-class solution, on termination the values immediately available to us are the class c **system throughput** X_c , the device k **queue length** Q_k and the class c **residence time at device k** $R_{c,k}$. We are also able to use Little's Law to calculate the class c **system response time** $N_c/X_c - Z_c$, the **average number of class c in system** $N_c - X_c \cdot Z_c$,

the class c throughput at device k $X_c \cdot V_{c,k}$, the class c utilisation at device k $X_c \cdot D_{c,k}$ and the class c queue length at device k $X_c \cdot R_{c,k}$ [4].

As this solution requires all feasible populations up to the desired population to be explored, e.g. Figure 2.3, these complex dependencies can grow rapidly. The time complexity of exact multi-class MVA is $O(C \cdot K \cdot \prod_{c=1}^C (N_c + 1))$, while the space complexity is $O(K \cdot \prod_{c=1, c \neq c_{\max}}^C (N_c + 1))$. The complex dependencies shown above lead to an extremely inefficient algorithm, making it impractical to compute the exact solution of multi-class networks for models with more than a few classes [4, 10].

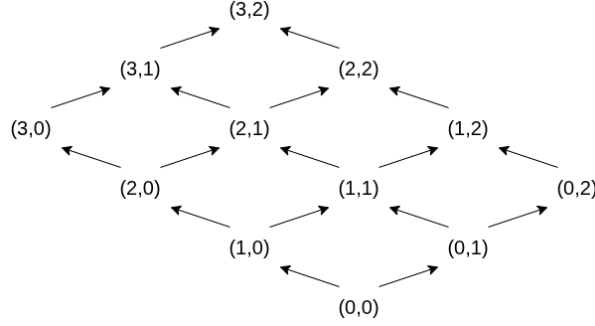


Figure 2.3: Precedence of exact multi-class MVA intermediate solutions.

Approximate Multi-Class MVA

Similarly to the single-class solution, an approximation for h can be used to maintain an accurate output whilst dramatically reducing space and time requirements, iterating around an estimate. A good approximation for h here is $h_c[Q_{1,k}(\mathbf{N}), \dots, Q_{C,k}(\mathbf{N})] \equiv \frac{N_c - 1}{N_c} \cdot Q_{c,k}(\mathbf{N}) + \sum_{j \neq c} Q_{j,k}(\mathbf{N})$. Intuitively, the removal of a customer from the network does not affect the placement of other classes, and reduces the queue lengths of its own class proportionally [4, 11].

Algorithm 2.4: Approximate MVA for multi-class models [4]

```

1:   $Q_{c,k}(\mathbf{N}) \leftarrow \frac{N_c}{K}$  for all  $c = 1, \dots, C$  and  $k = 1, \dots, K$ 
2:  repeat
3:     $Q_{c,k}^{\text{old}}(\mathbf{N}) \leftarrow Q_{c,k}(\mathbf{N})$  for all  $c, k$ 
4:    for  $c = 1$  to  $C$  do
5:      for  $k = 1$  to  $K$  do
6:         $A_{c,k}(\mathbf{N}) \leftarrow h_c[Q_{1,k}(\mathbf{N}), \dots, Q_{C,k}(\mathbf{N})]$ 
7:      end for
8:    end for
9:    for  $c = 1$  to  $C$  do
10:     for  $k = 1$  to  $K$  do
11:        $R_{c,k}(\mathbf{N}) \leftarrow \begin{cases} D_{c,k} & \text{(delay centres)} \\ D_{c,k}(1 + A_{c,k}(\mathbf{N})) & \text{(queueing centres)} \end{cases}$ 
12:     end for
13:   end for
14:   for  $c = 1$  to  $C$  do
15:      $X_c \leftarrow \frac{N_c}{Z_c + \sum_{k=1}^K R_{c,k}(\mathbf{N})}$ 
16:   end for
17:   for  $k = 1$  to  $K$  do

```

```

18:       $Q_{c,k}(\mathbf{N}) \leftarrow \sum_{c=1}^C X_c R_{c,k}(\mathbf{N})$ 
19:    end for
20:  until  $\max_{c,k} |Q_{c,k}(\mathbf{N}) - Q_{c,k}^{\text{old}}(\mathbf{N})| \leq \varepsilon$ 

```

The time complexity is reduced greatly, to a point where the population of the classes is not considered, $O(C \cdot K)$ per iteration. The space complexity is similarly reduced to $O(C \cdot K)$, as the algorithm only requires maintaining a solution for population $\mathbf{N}$ [4, 11].

2.3 Extended Queueing Networks

Classical queueing networks and Mean Value Analysis provide efficient solution techniques under product-form assumptions. However, these assumptions are frequently violated in distributed client-server systems. We must therefore assess the limitations of the classical approach and introduce the extensions that motivate Layered Queueing Networks.

2.3.1 Assumptions of Product-Form Queueing Networks

While product-form queueing networks enable efficient computation of performance measures, they rely on the underlying set of Baskett–Chandy–Muntz–Palacios (BCMP) product-form assumptions [5, 8]:

- Service centres are of one of these specific types:
 - A single server with a first-come-first-served (FCFS) queueing discipline, where the time a job needs at the server is exponentially distributed. The mean service times for all customers must be identical.
 - A single server with a last-come-first-served (LCFS), service-in-random-order (SIRO) or processor-sharing (PS) discipline. Service time distributions can be general (do not require the memoryless property), and different customers can have different service times.
 - A load-dependent first-come-first-served service centre, where the service rate is dependent on the number of customers at the station only. This subsumes the infinite-server (IS) as the special case $\mu(n) = n\mu$.
- The population of a closed product-form network is fixed.
- Customer routing is independent of the state of the network.

Despite these assumptions seeming restrictive, there are many models that violate them yet produce sufficiently accurate results. However, there are some cases in which product-form queueing networks (PFQNs) cannot be used directly. If these effects were ignored, we could overestimate the throughput of a model as we are not accounting for the time required to acquire the resources [5].

- **Systems with simultaneous resource possession.** The service rate at one centre may be dependent on other centres in the network. If a customer holds a resource at one station while waiting at another, for example, multi-programming lock contention or blocking remote procedure calls, the independence assumption breaks.
- **Systems with fork–join behaviour.** Forking generates new customers, whilst joining eliminates them, violating the fixed customer population and routing homogeneity assumptions.

This is where we consider the **method of surrogates**, a hierarchical decomposition strategy that accounts for the time needed to acquire simultaneously-held resources by splitting the

input model into submodels and iterating between them. Each submodel contains the explicit representation of one of the resources, and a delay centre representing the other. The waiting time for one of the resources is therefore solved and then used as the delay centre input in the other submodel. This process repeats, iterating between submodels until convergence [5].

2.3.2 Client–Server Models

To address the limitations of classical queueing networks, we now consider the adaptations that are tailored towards client–server architecture. These approaches all use hierarchical decomposition and the method of surrogates, breaking multi-tier client–server architecture into two-layer systems that are solved iteratively. Whilst the overall approach is similar, they take different implementation forms [5].

Stochastic Rendezvous Network Model

The Stochastic Rendezvous Network (SRVN) model [12] consists of an acyclic graph of clients and servers (referred to as *tasks*), used to model users and both hardware and software components, connected through remote procedure calls (where the clients are blocked until the server responds). The input model is broken down into a set of submodels that each consist of one server, the set of clients that call it, and surrogate delays for the other calls that the clients make. The clients are treated as unique customers, with a population based on the number of instances of the task. The model is then solved by applying one-step MVA to each of the submodels, adjusting the surrogate delays in the other submodels, and iterating until convergence. This allows the modelling of requests with *rendezvous* semantics, in which clients are blocked until servers reply, directly modelling synchronous remote procedure calls and nested service dependencies. Its *task*, *entry*, and *phase* abstractions become LQN’s central modelling vocabulary [13, 5].

Method of Layers

The Method of Layers (MOL) [14] explores hierarchical decomposition as a solution strategy for client–server systems, partitioning the system into a set of two-level MVA submodels. These are constructed by splitting the input model into a *hardware contention submodel*, containing all tasks as clients and devices as servers, and a *software contention submodel* which, considering only tasks, sorts the model into layers and constructs submodel n by using all tasks on level n as clients and all tasks at level $n + 1$ as servers. Surrogate delays are also introduced into the software contention submodels after servers that make a call to another task in the input model. Surrogate delays are introduced into the hardware contention submodel, with one for each task. These submodels are solved by iterating from submodel 1 to $N - 1$ until convergence. These measures are then used as the think and service times in the hardware model, which is solved, and the measures put back into the software submodels. This process repeats until convergence of both. This approach provides a scalable mechanism for analysing systems with synchronous interactions, where the surrogate delays act as approximates for blocking effects without requiring joint state modelling [5, 14].

Layered Queueing Networks

The Layered Queueing Network (LQN) [5] is the synthesis of the SRVN model and MOL. It inherits SRVN’s modelling vocabulary (tasks, entries, phases, synchronous and asynchronous calls) and MOL’s solution strategy (hierarchical decomposition into MVA submodels coupled by surrogate delays). LQNs explicitly represent software entities (tasks, entries, activities) with their limited concurrency and synchronous interactions, alongside hardware resources (processors). By structuring the model into layers and representing synchronous calls as rendezvous interactions, LQNs are able to capture blocking, nested service demands and contention effects that arise naturally in multi-tier systems [5, 6].

The increased modelling power of LQNs comes at the cost of increased computational and implementation complexity. LQN solvers rely on decomposition techniques, surrogate delays,

and approximate variants of MVA combined with fixed-point iteration [4, 5]. These techniques have been used in LQNS [6] and in LINE’s SolverLN [15], which support a wide range of modelling constructs such as interlocking, multi-server stations, and complex activity graphs.

2.4 Solving Layered Queueing Networks

We now move from the abstract LQN formalism to the algorithmic machinery that solves an LQN model. We introduce the input vocabulary the solvers consume, the decomposition pipeline that turns an LQN into a sequence of product-form submodels, the MVA variants that solve each submodel, further modelling constructs we see later, and the two specific solver implementations against which this project’s contribution is measured. We use the Layered Queueing Network Solver (LQNS) [6] as the primary reference implementation for LQN solution techniques, as it is the most widely used and feature-complete realisation of the LQN formalism [5].

2.4.1 LQN modelling basics

LQN’s solution apparatus is built on the Stochastic Rendezvous Network model. The inputs to the model are **tasks**, which represent hardware and software objects (analogous to service centres in flat queueing networks), **entries**, which define the per-task service demands (analogous to classes), and **phases** or **activities**, denoting the different stages of service within entries, allowing situations where remote procedure calls can respond before service is fully completed [5, 6].

Tasks

The LQN model classifies tasks in three ways [5]:

- **Pure clients / reference tasks** are tasks that only send messages. They are used to model input sources such as users or external request streams, as they cycle continuously and never accept calls themselves.
- **Pure servers** are tasks that only receive requests, used to model hardware servers such as processors or disks. They are the *leaves* of the call graph.
- **Active servers** are tasks that both accept and produce requests. These are typically software processes that handle an incoming call by making nested calls to lower-level tasks.

Tasks communicate with each other using the **send–receive–reply** paradigm, where the client is blocked until it receives a response from the server. Task service times are broken down into phases, with the task responding to the client at the end of phase one and then executing subsequent autonomous phases. Messages are passed to the server on a first-come-first-served basis, but the server can differentiate its actions based on the entry, analogous to classes in conventional queueing networks [5].

Tasks also carry a **multiplicity** attribute corresponding to the number of concurrent instances of the task. For a reference task this fixes the closed-network population, while for a non-reference task it sets the multi-server capacity at that task’s queue, making multiplicity the standard feature for expressing concurrency limits such as thread pools or worker processes.

Host and Task layers

An LQN solver can decompose a model into two types of submodels:

- A **host layer** contains one queue station per *processor* and aggregates the demands of all tasks hosted on that processor. The host layer is where physical hardware contention is modelled.
- A **task layer** contains the queue for one non-reference task as the server, and its direct callers as clients. The task layer is where *software* contention is modelled.

Inputs and outputs

The inputs for the LQN model, based around entries and phases, are the mean total **execution time** s_{ep} of entry e in phase p , the **squared coefficient of variation** c_{ep}^2 for a slice of that execution time, the **phase type** pt_{ep} (describing how requests are emitted to lower-level servers), the **external arrival stream** λ_{0e} of requests to entry e and the **mean number of requests** y_{edp} from entry e in phase p to entry d . The output we primarily care about is the **throughput** λ_e of entry e , however we can also calculate per-entry queue lengths, utilisations, and residence times [5].

2.4.2 Solving LQNs

Every LQN solver follows the same four-step pipeline, established in Franks’s 1999 thesis [5], formalised by LQNS [6], and inherited (with paradigm-flexibility extensions) by SolverLN [15]. We describe the pipeline at the algorithmic level here:

1. **Decompose the LQN into a sequence of PFQN submodels.** The LQN’s tasks and processors are partitioned into a set of host layers and task layers, where each submodel is a separate product-form queueing network whose stations are derived from the LQN’s tasks and processors. Different *layering strategies*, introduced below, produce different partitions of the same LQN. The strategy choice is itself an algorithmic parameter that the LQN solvers expose.
2. **Solve each submodel with an inner MVA kernel.** Once a submodel is produced as a PFQN, it can be handed to an MVA kernel, producing a complete set of **QN / UN / RN / TN outputs**.
3. **Couple between submodels by updating service and think times.** The outputs of each submodel are then used to update the *inputs* of the other submodels. A task’s residence time at its host layer becomes the service time it presents to its callers in the task layer. A caller task’s residence time at the callees becomes a contribution to the think time the caller presents to its own host.
4. **Iterate to a fixed point.** Steps 2 and 3 are repeated until every submodel’s outputs converge. The fixed point is the solution to the LQN.

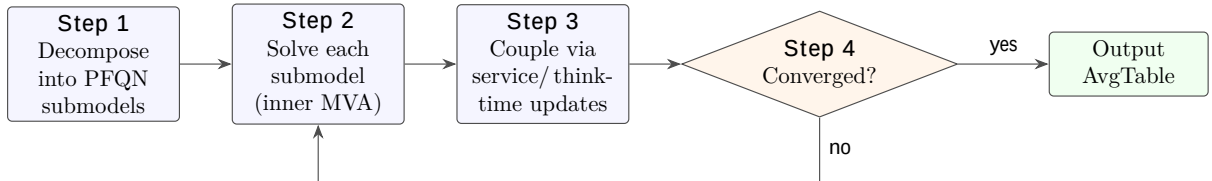


Figure 2.4: The standard four-step LQN solver pipeline.

This pipeline gives rise to two distinct kinds of iteration:

- The **inner loop** is the iteration *inside* a single MVA variant. This is the recursion over population in exact MVA, or the **repeat ... until** loop in approximate MVA. The inner loop runs at step 2 of the pipeline above. It produces one submodel’s outputs from its inputs.
- The **outer loop** is the iteration *over submodels*. This instruments the sweep through every submodel in steps 2 and 3, terminated by step 4’s convergence check. One outer iteration is one sweep through every submodel in the ensemble.

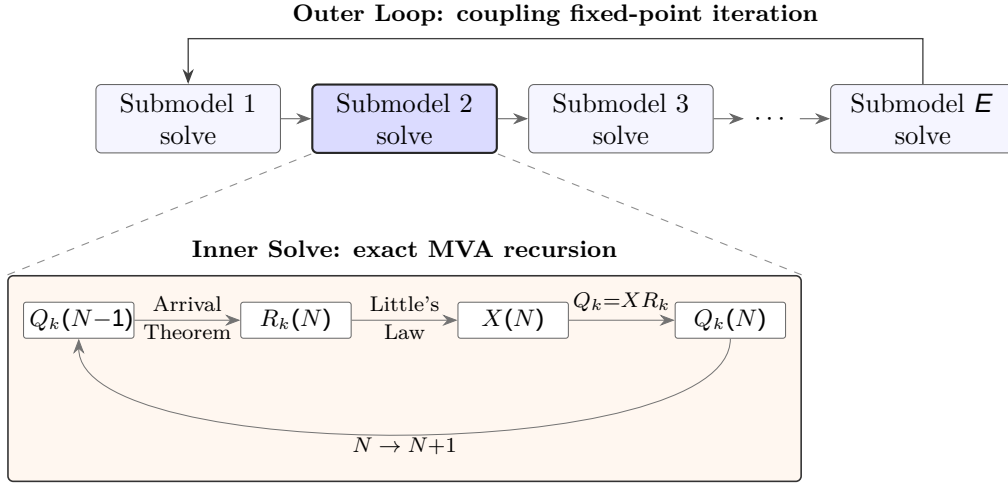


Figure 2.5: The outer fixed-point coupling loop wraps per-submodel inner MVA solves.

A worked example

Consider a small LQN with three elements: a reference task **T1** with closed population N and pure think time Z_{T1} , making one synchronous call per cycle to task **T2**, where **T2** has host demand D_{T2} on processor **P1**.

We decompose this into two PFQN submodels, each a closed product-form network sharing **T1** as its workload source but isolating a different bottleneck:

- **Host submodel P1** models contention at the processor. Its only queueing station is **P1**, with service demand D_{T2} . **T1** appears as an IS-delay client whose think time considers everything **T1** does when not at **P1**.
- **Task submodel T2** models contention at the task. Its only queueing station is **T2**'s service queue, and **T1** is the client. **T2**'s service time here is its residence time at **P1**, a value supplied by the host submodel.

The two submodels are coupled by **T2**'s residence time at **P1** flowing forward into Task Submodel **T2**, and **T1**'s response time at **T2** flows backwards into Host Submodel **P1**. One *outer iteration* is four steps:

1. **Solve Host Submodel P1** with MVA, returning R_{T2}^{P1} (pure demand plus some queueing).
2. **Push forward**: set **T2**'s service time in Task Submodel **T2** to R_{T2}^{P1} .
3. **Solve Task Submodel T2** with MVA, returning R_{T1}^{T2} (one full **T1**→**T2** call).
4. **Push back**: set **T1**'s think time in Host Submodel **P1** to $Z_{T1} + R_{T1}^{T2}$.

Convergence is declared when those scalars stop moving between successive sweeps.

2.4.3 MVA variants

Solving each submodel is parametric in which MVA variant we choose. No single variant is uniformly best: submodels differ in size, in whether stations have population-dependent service rates, and in how tight an accuracy/cost trade-off the caller can tolerate. We cover the MVA variants that arise in practice:

Exact MVA refers to the Reiser–Lavenberg recursion [10] from Section 2.2, giving the exact solution for any closed product-form network. Multi-class cost is exponential in the number of

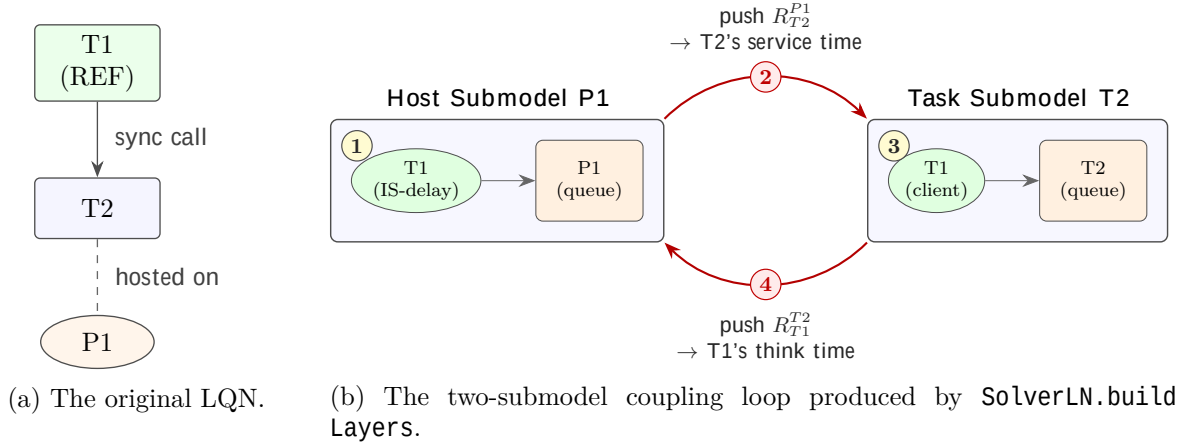


Figure 2.6: Decomposition and outer-loop coupling for the worked example.

classes, so it is competitive only when the lattice size, the population combinations that build the solution, $\text{prodN} = \prod_c (N_c + 1)$ is small.

Load-Dependent MVA (LD-MVA) [16] generalises exact MVA to stations with population-dependent service rates $\mu_k(j)$, conditioning the recursion on a queue-length distribution table $\pi(k | n)$ [4]. It is the standard kernel when a submodel contains a multi-server task whose rate must be modelled exactly.

Bard-Schweitzer Approximate MVA (AMVA) [11] replaces the exact arrival-instant relation with $A_k(N) \approx \frac{N-1}{N} Q_k(N)$, seen in Section 2.2, yielding a cheap $O(C \cdot K)$ -per-iteration fixed point on residence times.

Linearizer [17] refines Bard-Schweitzer with a second-order correction on the arrival-instant queue length, improving accuracy at moderate populations [18].

Seidmann's transform [19] folds a c -server station with demand L into a single-server station with demand L/c and a Z -shift of $L(c-1)/c$, with total queue length reconstructed via $Q_{\text{total}} = Q_{\text{BS}} + X \cdot L(c-1)/c$. It was extended to LQN solvers by Rolia [20], to keep the cheap single-server MVA kernels applicable when tasks and processors have multiplicity greater than one.

2.4.4 Further solver constructs

Mature solvers layer additional modelling constructs on top of the pipeline above, and are ideas we revisit throughout the contribution and evaluation chapters.

Activity graphs. Richer LQN models can be described by a directed precedence graph, whose nodes are activities and whose edges are typed. The edges can be a **sequence**, running activities in order, an **OR-fork**, branching probabilistically, an **AND-fork**, spawning parallel branches that an **AND-join** later synchronises, or a **loop**, repeating a body. A **reply** marks the point at which the caller's phase-1 synchronous wait ends, so any work after the reply is the autonomous phase-2 body that the processor still executes but the caller does not perceive. Because an AND-join completes only when its slowest branch does, the caller-perceived join delay is the *expected maximum* of the branch times. For exponentially distributed branches, this reduces to the inclusion-exclusion sum, though in practice solvers approximate it [5]. Activity graphs are what let an entry express conditional, parallel and iterative control flow rather than a single straight-line demand.

Under-relaxation. The outer fixed-point iteration is not guaranteed to converge monotonically and can instead oscillate, with a quantity overshooting on one sweep and undershooting on the next. Solvers mitigate this by moving a coupled input only a fraction α of the way to its newly computed value, $x \leftarrow (1 - \alpha) x_{\text{old}} + \alpha x_{\text{new}}$, rather than overwriting it outright.

Interlocking. When a server is reached along several call paths that share a common

ancestor task, the requests along those paths are not independent as a customer busy in one path cannot simultaneously be in any of the others. Treating the paths as independent would therefore overestimate the contention the server sees, inflating throughput values. Interlocking is the correction LQNS applies, discounting a server’s queueing by the fraction of its load that originates from a shared caller [5, 6]. Computing it requires walking the call graph to find the interlocked paths and their loads.

2.4.5 The Layered Queueing Network Solver (LQNS)

LQNS is the mature canonical reference implementation of an LQN solver, described formally in Franks et al. (TSE 2009) [6] and the LQNS user manual [21]. Its solution follows the four-step pipeline of Section 2.4.2 with the following specific choices.

Step 1: Decomposition into PFQN submodels. LQNS supports four layering strategies, each producing a different submodel partition of the same LQN:

- **Batched layering** is LQNS’s default strategy. Submodel l contains every task and processor at nesting level $l + 1$ as servers, with clients being any task that calls one of those servers [5].
- **Strict layering** is the Method of Layers strategy [14]. Software submodels are built top-down level by level, with all tasks at level L as clients and all tasks at level $l + 1$ as servers [5]. The hardware submodel has all tasks as clients and all devices as servers. Strict layering produces fewer but larger submodels than Batched.
- **Loose layering** is the strategy of the SRVN solver [12]. Each serving task or processor becomes its own submodel, with its callers as clients [5]. Loose layering produces the largest number of submodels but the simplest individual submodels, many of which are one-class, single-server PFQNs.
- **Squashed layering** collapses the entire LQN into a single submodel, with pure clients appearing as clients, pure servers as servers, and active servers appearing in both [5]. Squashed layering is rarely used in practice but is a useful reference point when comparing layering strategies, representing the extreme of having no decomposition.

Step 2: Inner MVA kernel. LQNS’s default inner kernel is the Linearizer [17], with Bard–Schweitzer as a fallback.

Step 3: Coupling. LQNS couples between submodels via **surrogate delays**, the central coupling mechanism of MOL [14]. A delay centre is inserted into each submodel to represent the time customers spend at servers in other submodels they call. The surrogate delay value is updated between iterations from the residence time of the callee submodel.

Step 4: Iteration. LQNS iterates submodels in a top-down then bottom-up *elevator* order, terminating when a per-station residence time delta falls below tolerance. This allows both think times to be propagated down and wait times to be propagated up equally, providing good estimates early on [5].

2.4.6 SolverLN: LINE’s layered solver

This project’s *primary* comparator is **SolverLN**, the LINE codebase’s layered solver, described formally in Casale, Gao, Niu and Zhu (2024) [15]. LINE is the open-source queueing analysis framework motivated by the observation that practitioner tooling has historically concentrated on just two paradigms, discrete-event simulation and approximate mean-value analysis, despite deeper stochastic-methods literature [22]. Its design response pairs a single modelling language with interoperable solver backends (MVA, NC, MAM, FLUID, CTMC, LDES, plus external dispatch to JMT and LQNS), and **SolverLN** inherits this multi-paradigm reach as per-layer solver dispatch, where LQNS commits to approximate MVA at every submodel [15].

Of the per-layer paradigms **SolverLN** can dispatch, this project engages with the MVA path specifically. **SolverLN** inherits significant machinery from LQNS by direct porting, the two solvers therefore belong to the same tool family. This project's bottom-up methodology is a deliberate response to this propagation pattern. Building from a base case rather than carrying

(OSL) drops Linearizer’s outer two-iteration loop and skips the redundant `core(N)` invocation that begins each subsequent call, instead warm-starting from the previous submodel’s solution. One-Step MVA (OSM) further removes Bard-Schweitzer’s `repeat ...until` entirely, letting the outer LQNS loop take its place. The evaluation is on the thirteen-layer Bookstore model from Israr, Woodside and Franks (2007) [24], with four solver configurations (Linearizer, OSL, Bard-Schweitzer, OSM) crossed with five submodel-layering strategies (Batched, Hardware-Software, MOL, Squashed, SRVN). Performance was evaluated on calls to `wait()`, representing one step of MVA. Under MOL layering, Linearizer made $87,852 \times 10^4$ calls while OSM only required 7×10^4 , a reduction of approximately $12,500\times$. Throughput results agreed to four decimal digits across the grid, whilst run time fell from 01:47 to 00:06. The paper notes however, that the number of outer iterations sometimes increased, confirming the paper’s own hypothesis that work formerly done by inner-loop convergence is absorbed into additional outer-loop sweeps.

Once again, however, this paper’s reductions focus on the inner AMVA loop rather than the outer loop. The motivating observation in the Conclusions section, *“Much of the work obtaining ‘accurate’ solutions during early iterations of the outer-most loop is wasted because the parameters for a given submodel are going to change anyway due to solutions to other submodels”* [23], is used to justify collapsing the inner loop, on the basis that the outer fixed-point iteration will invalidate any over-precise submodel solution. The outer loop itself, however, is left unchanged.

We therefore look to see if there is a similar reduction we can achieve on the outer loop from a stripped back version of the solver. Whilst their reduction asks whether the inner AMVA loop’s full convergence is necessary given the outer loop’s re-run, we ask whether specific outer-loop machineries are necessary, given the structure of submodel coupling.

Chapter 3

Contribution

This chapter presents what was built and why. The artefact produced is an LQN solver named `SolverLNSimple`. The established comparators (LINE’s `SolverLN` [15] and LQNS V5 [6, 21]) are mature, general-purpose solvers that have become bloated with machinery over time to handle every possible case, including interlock corrections, an `iter_min` floor and moving-average convergence. `SolverLNSimple` is built the other way around. Each piece of machinery added responds directly to a specific case of model that the previous design could not handle. The result, though not as general-purpose as the existing solvers, is focused and efficient. The result is a solver that, on the closed-LQN models in our evaluation corpus, converges in fewer outer iterations than either comparator, and does so in less clock time, providing evidence that some iteration cost of the existing solvers is structural rather than necessary. The chapter develops six key features, each motivated by the limitations of the solver on a set of models, beginning with a minimal LQN solver (Sections 3.1–3.7).

3.1 Coupling on the ensemble

The starting point is the smallest LQN solver that works at all. It takes a `LayeredNetwork`, decomposes it into an ensemble of PFQN submodels via `model.getEnsemble()` (which internally calls `SolverLN.buildLayers`), and wraps a coupling-iteration loop, `iterateCoupledMva`, around a PFQN MVA kernel, `Pfqn_mvamsKt.pfqn_mvams`. The ensemble building and kernel is delegated deliberately as we want to compare *outer* iteration counts on like terms with `SolverLN` and LQNS. Working with a different ensemble would make comparisons less like-for-like, and re-implementing MVA would only reproduce the same numerical results. The first question this minimal solver must answer is how to couple the submodels.

3.1.1 The minimal solver and its first coupling

`SolverLN.buildLayers(model)` returns one PFQN sub-`Network` per server, where each host layer server is named `P:<processorName>` and each non-REF task layer server `T:<taskName>`, with reference tasks deliberately folded into the `Clients` delays of their callees’ host layers rather than given their own layer. Each submodel is therefore a standard product-form closed network, with a Delay node for the clients and a queueing node for the server. In each sweep, the solver solves every layer with `pfqn_mvams`, which is a multi-class MVA kernel (a single class is fed as $R = 1$, producing correct single-class results while leaving room to extend to the multi-class case later), then mutates Delay and Queue service times to propagate load between layers. We call `iterateCoupledMva(maxIters, tol)` with a per-layer throughput-delta tolerance, converging when the maximum delta falls below it.

The first coupling attempt to match nodes in submodels by name. A `nameIndexMap : Map<String, List<int[]>`, built on construction, maps each node name to the (layer, node Index) pairs where it appears, and two update methods, `updateThinkTimeFromUtilization` (Little’s-law identity $z = \text{pop} \cdot (1 - U) / X$) and `updateServiceTimeFromResidence` ($Q /$

X), write through it. It is the natural first attempt, but naive as it does not comply with what `SolverLN.buildLayers` produces.

3.1.2 Failure

Name-based coupling collapses because `SolverLN.buildLayers` does not preserve node names. It prefixes every server with `T:` or `P:` and groups all clients of the server into a single node named `Clients`. Solving with the naive solver would result in `nameIndexMap` matching only on `Clients`, `updateThinkTimeFromUtilization` zeroing the Delay think times because `Clients` IS-station utilisation ≈ 1 drives $z = N \cdot (1 - U) / X$ to zero, and `updateServiceTimeFromResidence` would then collapse every demand to `Exp.fitMean(0)`. The coupling channel has to be rebuilt around what `buildLayers` actually shares.

3.1.3 Response

We therefore discard name matching and re-couple submodels through the `T:` / `P:` queue-name prefixes from `SolverLN.buildLayers`. Coupling is driven instead by `queueNameToLayer` and the `MvaUtils` helpers. Similarly to the naive solver, the $Z_{\text{callee_host}} = (N - Q_{\text{server}}) / X$ formula updates a host layer's `Clients` think time and the $R_{\text{task}} = Q / X$ formula sets a task layer's service time.

The prefix abstraction

Four fields are introduced at the constructor that make the prefix convention usable. `baseClientsThink : double[N_LAYERS]` captures the initial `Clients` think time per layer before any coupling, `queueNameToLayer : Map<String, Integer>` maps each task / processor queue name to the index of the ensemble layer that contains it to enable layer lookup in the coupling, and auxiliary maps `taskToProcessor` and `processorToTasks` are derived from the LQN model directly to traverse the host/task relationship without having to re-walk the call graph. For modularity, we move all per-layer MVA matrix construction to a new class `MvaUtils`.

The $Z_{\text{callee_host}}$ formula

When `SolverLNSimple` solves task `T`'s layer, the host layer the processor `T` runs on needs its `Clients` think updated to reflect the time `T` spends away from the host server. By Little's law on a closed M/M/c sub-network, we get $Z_{\text{callee_host}} = (N - Q_{\text{server}}) / X$, a formula derived from $n_{\text{jobs}} \cdot |1 - \text{util}| / t_{\text{put}} - \text{thinktime}$. We also find that `pfqn_mvamd`, called from `pfqn_mvams`, returns $U = 1 - P(\text{idle})$, representing its busy probability rather than per-server utilisation. We therefore correct this by computing $u_{\text{per_server}} = X \cdot D / c$ directly.

3.1.4 Rejected alternatives

An initial ambitious approach was a complete chain-walking implementation that attempted entire solution rather than building only what solves an extremely basic LQN model, falling into exactly what we said this project should not do. The attempt consisted of parsing the LQN model directly, building task layers from the call graph, and running three MVA cases inline (IS for $c \geq N$, exact FCFS for $c = 1$, Bard-Schweitzer for $c > 1$). This approach failed to account for processor contention as we only considered task layers, plus many other factors, which proved this was a much more complex solution than what we required at this stage, diverging from the motivation of the project.

3.1.5 What ensemble coupling provides

Ensemble coupling was tested on two-layer and three-layer chain models. Whilst it is not bug-free, it finds correct results for some simple given cases, and allows us to explore further. Graph-derived coupling (Section 3.2) closes these bugs, narrowing the gap to `SolverLN` towards a competent, efficient solver.

The structural lessons are that we should look to consider how the tools given to us work rather than attempting an approach, and that we need to remain mindful of the project's scope

and the specific task at hand, rather than attempting complex, wholesale fixes. The coupling channel is dictated by what `SolverLN.buildLayers` shares across layers (the `T:` / `P:` server prefixes and a single `Clients` class) and therefore must be indexed by the prefix. Delegating the modelling and MVA to the ensemble allowed us to produce the smallest possible structure that lets the coupling formulas express their additive contributions to a fixed pre-coupling baseline.

3.2 Graph-derived coupling

When the prefix coupling solve was fed richer LQN chains, the solver produced incorrect results. Each symptom traced to a different blind spot of the prefix abstraction: the static `baseClientsThink[callerTaskLayer]` baseline read zero when the caller layer was Immediate, the $(N - Q_{\text{server}})/X$ coupling formula could divide by a throughput collapsing to zero which in turn propagates NaN/Infinity through subsequent `Clients` writes, and `getRootRefThinkTime` would walk the call chain all the way to the REF task as we had not considered any intermediate think times. The response was to stop deriving coupling targets from ensemble-layer indexing as this led to too many assumptions, and instead read them from the `LayeredNetwork` graph itself, via root-REF walks, caller-chain demand sums and subtree demand checks, providing a more natural approach.

3.2.1 Failure

To first check the prefix coupling approach and investigate any blind spots, a suite of simple LQN models was constructed by hand. When a failure was observed (`SolverLNSimple`'s result deviates from `SolverLN`), we would attempt to reproduce and pin down the exact point of failure. With the suite in place, the blind spots fall into three classes, seen in Figure 3.1 and described below.

Depth ≥ 3 chains with Immediate intermediate activities. On models with intermediate tasks with Immediate activities, `baseClientsThink[callerTaskLayer]` is initially zero, while the intermediate layer has $D = 0$ and zero caller think time. The callee's MVA solve therefore sees no upstream load and converges to a meaningless fixed point.

Immediate-leaf subtrees. For a model where every task in some sub-tree has `getHostDemandMean()` ≈ 0 , the $(N - Q_{\text{server}})/X$ formula divides by an X that is essentially zero, producing Infinity/NaNs that propagate through the next sweep's `Clients-think` writes.

Intermediate tasks with non-zero think times. On models where intermediate tasks have their own think time, any REF-walking lookup, such as `getRootRefThinkTime()`, terminates at the root REF, ignoring intermediate think times along the chain.

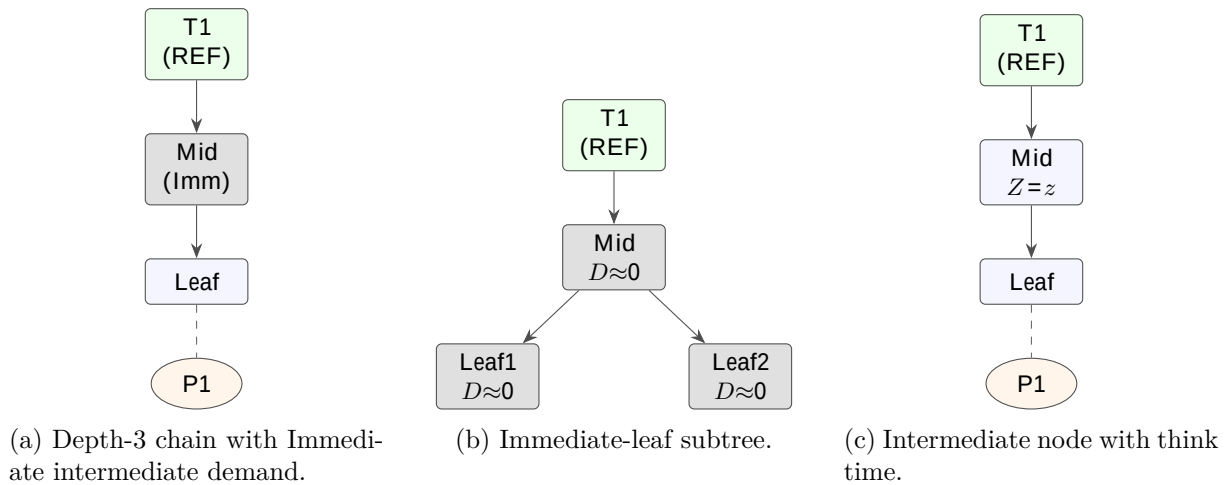


Figure 3.1: The three blind spots motivating graph-derived coupling.

The primary observation is that the prefix coupling creates an abstraction of the real inform-

ation that lives in the LQN graph based on false assumptions. If we access this directly, we can gain a deeper and correct understanding of the system’s behaviour, with no loss of information.

3.2.2 Response

The updated design replaces `baseClientsThink[callerTaskLayer]` indexing with a structured set of helpers that take task names as inputs and read the `LayeredNetwork` graph directly.

Chain-walk root-REF lookup

The first chain-walking helper is `findRootThink(int startLayer)`. The previous design read the immediate caller’s think directly as `baseClientsThink[callerTaskLayer]`, a single index one hop up. This is only valid in depth 2 fixtures, when the immediate caller is the root REF, but fails when an Immediate intermediate sits between the REF and the callee, returning 0 and causing the callee to see no upstream load. `findRootThink` therefore replaces this with a walk up the chain until a R: REF task layer is reached. We also implement a `[task → caller-service]` injection for caller task layers, the caller’s service demand can be overwritten and the callee response reaches the caller’s MVA solve. We also ensure `taskResidT` is reported zero for Immediate non-REF tasks, ensuring the propagated callee response is not reported as the intermediate’s own service. These changes solve the $\text{depth} \geq 3$ with Immediate intermediates category, but the more important headline is the shift from a single-index lookup to walking the `LayeredNetwork` graph itself.

Multi-call support

Previously, we assumed each activity can only make one call for simplicity, however it is possible for a single activity to invoke multiple calls to its target each visit. The change adds a recursive `resolveTaskResponseTime` helper that reconstructs task response time through the call chain, multiplying through by the number of calls, `callMean`, at each step. We can then use `propagatedTaskResp = R_task_total × syncCallMean` in the caller-side coupling.

Structural updates above the throughput guard

We then look to close the Infinity/NaN-propagation failure. We do so with `hasServerDemandInSubtree(calleeTask)`, another recursive callee walk, that gates `Z_callee_host` such that if the subtree has no server demand, we use `calleeThinkTime` directly, instead of the `max(calleeThinkTime, (N - Q_server)/X)` form. Our approach is to make sure that clients think updates take place even when we get non-finite solutions so we can exit the degenerate state, providing a simple and structural guarantee.

Intermediate-think coupling

To handle the case of an intermediate task with a think time, we must use the formula `Z_callee_clients = max(refChainThink, directCallerThink)`, where `refChainThink` sums root think time and caller demand, while `directCallerThink` sums caller task think time and its local demand. This recognises that the callee’s Clients think time must equal the bottleneck’s think time, considering both the REF chain and the intermediate. A similar formula handles intermediate callees for host updates: `Z_callee_host = max(calleeThinkTime, callerChainZ) + (R_task_total - calleeLocalDemand)`, where the subtraction avoids double-counting because the host server already provides `calleeLocalDemand`.

3.2.3 Rejected alternatives

Several fixes were attempted before consolidating on the final graph-derived design, all ending up superseded or at a dead-end. Index based fixes such as layer index in `baseClientsThink[callerTaskLayer] → baseClientsThink[l]`, per-layer-N overrides in `MvaUtils.buildN` and `taskCalledByTask` reverse maps were attempted, trying local fixes. Propagation-based fixes were also looked at, including two-phase Jacobi / elevator sweeps, `pendingUpdates` deferred-coupling buffering and a `pendingCalleeUpdates`-array implementation, exploring if the current coupling behaviour was truly sound. The significance of these rejected experiments is that many local

fixes to the prefix design were tried before recognising that the abstraction itself had to be replaced.

3.2.4 What graph-derived coupling provides

Graph-derived coupling enables the handling of $\text{depth} \geq 3$ chains with Immediate intermediate activities, Immediate-leaf subtrees without NaN propagation, intermediate tasks with their own think times and single-class chains of arbitrary depth. The coupling derivation has now moved from the name matching baseline, to prefix indexing, and now to graph traversal (functions in `LqnGraph` helper), emerged from the problem at hand and became the foundation for the rest of the solver. We also learn the value of structural fixes over local ones, creating a robust solution that can handle failures and allows the solver to be extended easily.

3.3 Multi-class support

When the single-class machinery so far developed met a multi-class LQN topology, where multiple reference tasks can call shared tasks, it could not represent the per-caller demands in any single class, aggregating instead. The response was to handle each class directly. We feed multi-class demands per layer into `pfqn_mvams`, cap per-T:-layer populations to the minimum effective N, split `callMean` into its two semantically distinct uses, and mutate the ensemble’s host-layer class topology to recover the per-caller information.

3.3.1 Failure

Our first action is once again to create groups of models, this time specifically multi-class ones, that fail. We determine Category A (multiple REF tasks that call their own submodels, inducing no sharing), Category B (multiple REF tasks calling a shared server with the same demand) and Category C (multiple REF tasks calling a shared m-server with asymmetric demands) as the three categories that will enable us to diagnose the system, shown in Figure 3.2.

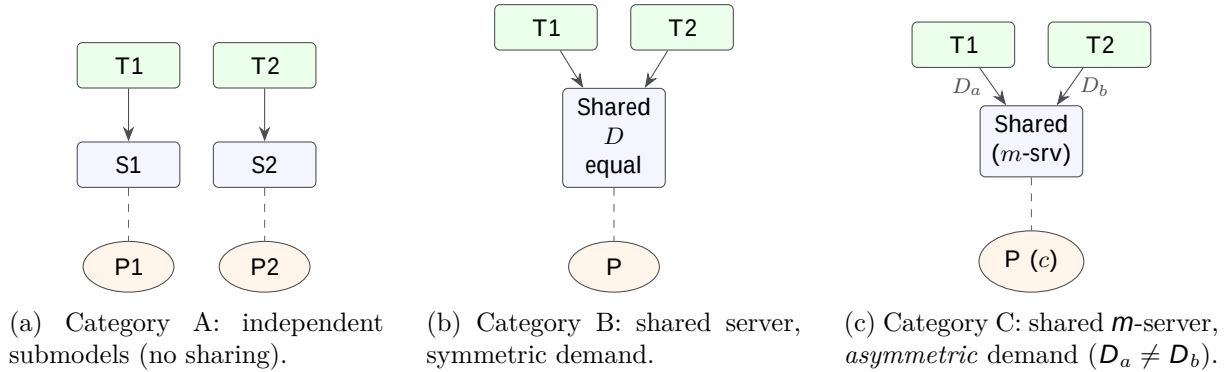


Figure 3.2: The multi-class fixture categories.

These fixtures expose two key blind spots. We use *single-class matrix builders* to call MVA with, following our minimal solution paradigm where up to this point we only had one class, which is what the solver uses `MvaUtils.getMainClass` in the matrix builders. We also only consider a single aggregate class in our models, which works when we are met with multiple classes with equal demands, but falls apart as soon as we see *asymmetric per-caller demands*, losing the ability to recover per-caller results.

3.3.2 Response

Multi-class per-layer demands

Our previous design consideration of choosing to delegate MVA to `pfqn_mvams` allows us to feed multi-class demands into the MVA naturally, as it already supports $R > 1$ ($M \times R$ demand, $1 \times R$ population, $1 \times R$ think-time).

We obtain the multi-class demands by iterating the ensemble layers and feeding the server demands into the MVA matrix builders. For each layer we identify `tasksOnServer` (the single task for a `T:` layer, every task on the processor for a `P:` server), assigning a service demand to each closed class via one of two rules. An aggregate `T:xxx` class on a `P:` server represents the hosted task, where its demand is the caller-multiplicity-weighted mean host demand. For per-caller `R:caller` classes, we sum `callMean × serverAct.getHostDemandMean()` over sync calls to server entries.

`MvaUtils` extends to multi-class by introducing `getClosedClasses` to return all closed classes with `N > 0` and modifying `buildDemandMatrix` to return $M \times R$, and `buildN` and `buildThinkTimeMatrix` to return $1 \times R$. The `iterateCoupledMva` coupling loop is updated to iterate over all classes, tracking convergence via per-class-per-layer throughput, implementing per-class Clients-think writes, per-class `R_task_r` in the `[task → host]` loop and per-class `R_proc_r` in the host-submodel branch. Category A correctness (no sharing, block-diagonal demand matrices) falls out as the degenerate case of the multi-class machinery.

Multiplicity per-layer cap

Whilst not explicitly a multi-class issue, the multi-class fixtures helped us discover an issue with the solver's population cap logic. We implement `enforceMaxMult`. It sums caller-class populations per `T:` layer into a `taskEffectiveN` map, then for each `P:` layer it finds the minimum effective `N` across its hosted `T:` classes, and caps `T:` classes whose `getNumberOfJobs()` exceeds that minimum. This ensures that bottleneck demands are properly managed, and non-bottleneck layers are not capped. We do this without touching the processor server count or the `REF` class directly, so no class-instance mutation can leak across layers. This is because `LayeredNetwork.getEnsemble()` shares `JobClass` instances across layers, meaning any solver that needs per-layer class semantics must either avoid mutating classes or add new classes rather than mutating existing ones.

Per-class QLen attribution

The third sub-move addresses the residual after the first two: `taskQLen.put(taskName, q)` stores only class-0 QLen, so all activity queue lengths are wrongly reported as the class-0 QLen. The deeper structural diagnosis is that "callMean" was being asked to do two conflicting jobs (Throughput scaling requires the *max* per caller to convert a leaf task's per-visit throughput into its per-call rate, and *QLen apportionment* needs the *sum* across callers to fairly split the shared queue's length across callers). A single overloaded function cannot serve both sites.

To implement this fix, we first split `getTotalInboundCallMean` into `getTotalInboundCallMean` and `getSumInboundCallMean`, plus `getCallerTotalCallMean(callerTask, targetTask)` for per-caller-pair lookups. Secondly, a parallel map `taskClassQLen : Map<String, Map<String, Double>>` tracks per-caller QLen at each task, populated from the per-class MVA `Q[serverIdx, rc]` rather than a pooled `taskQLen[taskName]`. Finally, `applyRefTaskInheritance`'s signature is extended to take `taskClassQLen`, and the `q_callee` apportionment becomes `q_callee += (m / callerTotalCM) × classSpecificQ` when class-specific QLen is available, replacing the naive `q_callee += classSpecificQ`.

Mutating the ensemble's class topology

The fourth sub-move mutates the ensemble's class topology rather than refining coupling formulas that operate on it. The trigger is Category C (asymmetric per-caller demands), diagnosing that no coupling formula can recover the per-caller information from a single aggregate class. We first attempted a mini PFQN solver that ran in-place on the ensemble at every iteration, calculating the necessary values. Although this produced the correct results, it required a lot of special-case machinery and side-channel computation, which is why we opted to represent the per-caller demands as new classes directly in the ensemble, adding per-caller `R:` classes to the ensemble at construction, disabling the aggregate.

The new method walks each `P:` host layer, where the hosted task is INF-scheduled,

called by multiple REF tasks and does not make sync calls itself. For each such layer it adds new `ClosedClass("R:" + callerName, mult, clientDelay)` instances marked `setReferenceClass(true)`, disables the aggregate class via `setPopulation(0)` and `Disabled.getInstance()` on both Clients and server queues, assigns per-class `Exp.fitMean(callerThink)` at Clients and `Exp.fitMean(callerDemand)` at the server, and rebuilds the routing matrix `Clients  $\leftrightarrow$  Queue` for every class.

`perCallerHostDemand : Map<String, Map<String, Double>` and `aggregateHostDemand : Map<String, Double>` capture the demands collected during the constructor's per-class block. The `iterateCoupledMva` loop adds an early branch that feeds per-class `R_proc_r` directly into the hosted task's server and skips the `Z_callee_host` update for rebuilt layers. The result extraction has to be extended too, adding a per-rebuilt-layer aggregation branch summing `q_r / x_r` over per-caller classes back to the hosted task.

This is the only capability that mutates ensemble structure directly, rather than complicating the coupling formulas.

3.3.3 Rejected alternatives

We attempted many alternatives to the final design, each rejected for a different reason. Multi-class support is where the ensemble's class topology first proved misaligned with the LQN's class structure, forcing us to mutate it to recover the necessary information. The rejected alternatives include:

We considered `solveIndependentChains` as a Category A initial solution and performance optimisation, however decided against this as it was not necessary since we can solve multi-class models natively with `pfqn_mvams`. Instead, this would add extra code paths, bloating the solution and going against the spirit of this project.

Two fixes we attempted were using the LQN struct's `maxmult` array to cap populations directly, and walking the caller chain to compute an effective multiplicity. Both hit the architectural blocker of `LayeredNetwork.getEnsemble()` returning layers sharing identical `JobClass` object instances across layers. When `setPopulation()` was called on a class, it would cause unintended cross-layer population mutations, which motivated our non-mutating `enforceMaxMult` approach.

Each alternative approach was implemented enough to produce numerical results before being reverted. After rejecting the mini PFQN approach, we first looked at a per-class scaling factor `D_r / D_agg` at the coupling site. We then looked at modifying `MvaUtils.callMVA` to dispatch to `pfqn_bs` when all non-Delay rows have a single server. Finally, we replaced the rebuilt-layer skip-guard with a per-class dynamic Z update in the FCFS coupling branch. However, numerical errors remained in each of these approaches, which is why we chose the more ambitious structural mutation of the ensemble's class topology.

3.3.4 What multi-class support provides

Multi-class support handles multi-class LQNs within a 4.9% tolerance. This residual is attributable to `SolverLN`'s `N=0` structural classes plus class-switching routing, which our simple solver deliberately does not replicate for simplicity. Across its four sub-moves, multi-class support teaches distinct lessons. Firstly, adding multi-class support did not require new MVA machinery, instead just requiring richer inputs for the MVA kernel. This is a recurring pattern across the build, with the project's contribution being structure, rather than re-inventing the algorithms that LINE already gets right. Secondly, we learn that we should not always pick the first correct solution as it may not align with the goals of the project, as seen with the mini-PFQN approach. Finally, we learnt that whilst the ensemble is a powerful abstraction, it is not perfect. We must still look at ways to read the LQN graph directly, or mutate the ensemble's structure such that we can handle cases in a simpler way than `SolverLN`. The decision not to follow `SolverLN`'s approach directly is the project being honest about where its discipline matters and what "errors" are real and what are artefacts of the solver, considering the efficiency-accuracy trade-off.

3.4 An MVA kernel for LQN-shaped inputs

When the multi-class solver was run on the topologies that arise in larger LQN ensembles, such as large population models or deep chains, the `jline.api.pfqnlld.Pfqnmvald` kernel, called from `Pfqnmvamskt.pfqnmvams`, was both quadratically slow (roughly 60 s total test runtime at $N = 200$, decomposing into approximately 2.5 s per layer solve across all sweeps and layers) and numerically wrong on IS topologies (throughput collapsed at $N = 100$). The response was three layered moves, each responding to a different flaw. These were re-implementing the LD recursion with primitive `double[]` arrays in a local `Mvald` to prevent slow execution from matrix multiplication sparseness, an AMVA path for `prodN > exactLatticeMax = 500` in `MvaAmva` for large models, and partitioning IS stations before MVA dispatch to contribute to Z rather than to the lattice. This is the build’s headline performance work.

3.4.1 Failure

The opening trigger is a network with $N=200$ single-class multi-server model. With the current solver, each layer solve takes roughly 2.5 s, and across all iterations of all layers the full test runs in about 60s. The kernel causing the slow execution is `jline.api.pfqnlld.Pfqnmvald`, the load-dependent MVA implementation. Its `pi` table (the marginal-queue-length probabilities $pi(k|n)$ indexed by (k, n) , used in the LD recursion) is allocated as a sparse-CSC `Matrix`. The MVA recursion accesses the table densely, reading every (k, n) cell at step. These dense $O(N^2)$ `get/set` operations against a sparse-CSC matrix is the bottleneck. The second flaw is numerical and surfaces independently, where we note that instability starts at $N = 20$ in the 2-station IS topology, leading to collapsed throughputs. Both flaws point to the fact that `Pfqnmvald` is the wrong kernel for some LQN-shaped inputs, as it is too slow for dense large- N accesses, and wrong on IS-only inputs.

3.4.2 Response

The three moves respond to the two flaws by fixing the representation, adding an approximate path, and partitioning IS stations before dispatch. Each move is intentionally narrow, leading to a clean solution.

Re-implementing LD recursion with primitive arrays

The first move introduces `Mvald.java` in `ln_simple/mva/`. `Mvald` is a faithful, numerically identical re-implementation of `jline.api.pfqnlld.pfqnmvald` (same recursion, same accumulation order, same stabilisation rule), with the difference being that we are using primitive `double[]` arrays for the marginal queue-length probability tables $pi(k|n)$ and per-population throughputs XS instead of sparse-CSC matrices. We index the flat `double[]` array via $(k * (prodN+1)) + n$, where $prodN = \prod_r (N[r]+1)$. This allows the dense access pattern in the LD recursion runs at primitive-array speed rather than at sparse-CSC speed. `MvaInputs.callMVA` is rewritten to route the closed multi-server case through `Mvald` instead of the call to `Pfqnmvamskt.pfqnmvams`, as this was when it was routed to `pfqnmvald`. Single-server topologies therefore still use the existing kernel for simplicity.

AMVA dispatch for large prodN

`Mvald`’s $O(prodN \cdot Nsum)$ lattice enumeration is still expensive at very large N . The next move adds an approximate MVA path, known to be a more efficient solution to traditional MVA whilst still providing accurate results for sufficient N . We perform a threshold sweep over 12 models $\times$ 6 thresholds as a rigorous evaluation for when we should dispatch to AMVA. Six concrete alternatives were evaluated and rejected before settling: three candidate kernels (Naive Schweitzer, Krzesinski Linearizer, plain Bard-Schweitzer) and two global policies (all-LD, all-AMVA) were rejected in favour of the $prodN \leq 500$ threshold.

Naive Schweitzer multi-server BS ($W = L \cdot (1 + Q \cdot (N - 1) / (N \cdot c))$) resulted in a 17–50% error versus `SolverLN`, making it too inaccurate to be a candidate. *Krzesinski Linearizer*

(*pfqn_linearizermskt*) ($PB = 2 \cdot \text{sum}Q / (\text{pop} + 1 - \text{nserver})$) goes negative when $\text{nserver} \approx \text{ntot}$, which the project’s specific input shape of INF-folded inputs forced. *Plain Bard-Schweitzer* (*pfqn_bskt*) returned per-server Q not total, meaning the patch would need a wrapper, so we abandoned in favour of an in-place implementation.

Understanding **SolverLN**’s own multi-server AMVA path (`jline/solvers/mva/handlers/Solver_amva.kt`), we gathered that it uses the Seidmann/Rolia transform explicitly. It applies $L' = L/c$ and $Z' = Z + \sum L \cdot (c-1)/c$, then plain BS. We therefore implement the **MvaAmva** class with the same logic, once again in primitive `double[]` form. The transformation gives us $L'[i,r] = L[i,r] / c[i]$ $\Delta Z[r] = \sum_q L[q,r] \cdot (c[q] - 1) / c[q]$. We then use Bard-Schweitzer iteration on the transformed inputs $(L', Z + \Delta Z)$, followed by reconstruction of the total queue length $Q_{\text{total}}[i,r] = Q_{\text{BS}}[q,r] + X[r] \cdot L[i,r] \cdot (c[i] - 1) / c[i]$. The project’s discipline, match **SolverLN**’s idiom where it is right and depart where it has reason to, is once again on display.

Partition IS stations before dispatch

After resolving the performance issues, the final move addresses the numerical flaw we have been ignoring.

Our initial solution was to not create **T**: layers for INF-scheduled tasks at all, restructuring how the ensemble feeds into the solver. As they contribute no service demand, we are able to do this, propagating their effects via the per-caller host-layer mechanism. Whilst this solution matches mature solvers such as LQNS, it required a large restructuring of the solver, so we looked for a localised fix.

Once again, we looked at what **SolverLN** does. **SolverLN** partitions stations by **Sched Strategy** before passing them to MVA, such that INF servers contribute their service demand to the effective Z and are reconstructed post-MVA, so that the MVA recursion never sees them. We rewrite `MvaInputs.callMVA` to do just that. For each station with $S[i] \geq \text{ntot}$ (effectively IS), we mark `isInf[i] = true`, fold $L[i,r]$ into $z\text{Eff}[r]$ and zero $l\text{Eff}[i,r]$. If every station is IS, we can short-circuit via `analyticalISResult`: $X[r] = N[r] / Z_{\text{eff}}[r]$, $Q[i,r] = X[r] \cdot L_{\text{original}}[i,r]$, $U[i] = 0$. Otherwise, we dispatch to **MvaAmva** or **MvaLd** on the non-IS subset only, patching the IS rows of Q back in via `rebuildInfStationQ` post-MVA.

3.4.3 What the MVA kernel provides

The quantitative result from this work is huge. Our **N=200** model drops from approximately 60 s per test run to just 0.1 s, a $600\times$ speedup, from re-implementing `Pfqn_mvald`’s recursion with primitive arrays. The AMVA dispatch gives us a further speedup for large models, and the IS partitioning fixes a numerical issue. Figure 3.3 plots all three effects: the primitive-array **MvaLd** and **MvaAmva** kernels against their sparse-**Matrix** equivalents, and the `prodN=500` dispatch threshold between them.

One lesson we learnt was that sometimes the LINE abstractions we use actually hide representational costs. Despite sparse-CSC being a sensible library default, it was the wrong choice on a workload it was not designed for. Another lesson was that it can be better to evaluate heuristic thresholds empirically rather than arguing from theory, as competing error modes provide a non-monotone trade-off. Finally, when a mature reference solver implementation is available, it pays to try simpler approaches first and measure where they fail before reading the reference’s source. These specific failure modes reveal which constraints the reference’s design is actually solving, and what fixes need to be made, so the resulting code can provide the correct minimal solution.

3.5 Fan-out callers

This capability responds to the remaining fixture surfaces the build had not yet exercised. The headline failure is fan-out: when the solver meets LQN topologies where a single caller makes synchronous calls to multiple distinct callees, the coupling iteration enters a two-state limit

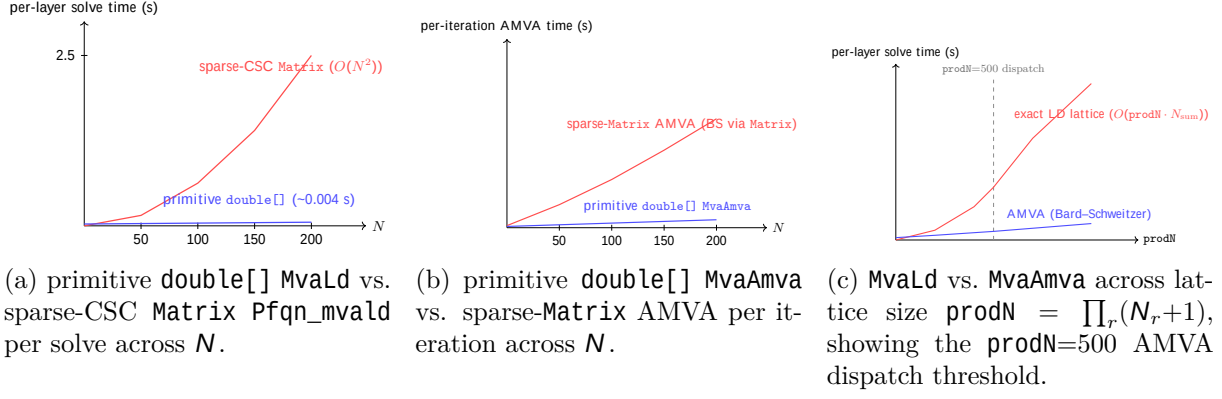


Figure 3.3: Motivation for AMVA dispatch and primitive MvaLd/MvaAmva.

cycle, converging in 3 iterations to a fully-saturated state. The fix adds time-at-siblings into the caller's Clients think, plus a conditional under-relaxation $Z_RELAX_ALPHA = 0.5$ gated on `LqnGraph.callerFansOut` that keeps single-callee chains un-damped. This work also calibrates the multi-class machinery (Section 3.3) against fixtures that intentionally attempt to result in a high iteration count. These combine multi-class callees with multi-entry callers.

3.5.1 Failure

Up to this point, fan-out structures were not a pattern we considered. With the current solver, the model saturates as tasks get inflated `R_proc` from host layers written to their demand via `writeHostResponseToTaskLayerServer`. The missing fix being that on a fan-out caller, the caller's effective Clients think time must include time spent at each sibling callee, but currently only returns `max(refChainThink, directThink)`, trapping the iteration at a false fixed point.

3.5.2 Response

The fix starts with `LqnGraph.callerFansOut(model, taskName)`, iterating the task's activities, collecting distinct callee task names, and returns true when the set has more than one element. `SolverLNSimple.computeSiblingCalleeBlocking(callerTask, excludedCallee)` then iterates the caller's activities and accumulates `callMean × R(sibling)`, reading from `taskSojournCache`. The term is used in `computeCallerClientsThink`, `propagateHostLayerCoupling`'s per-class `newZ`, and `pushRefTaskHostResponseToCalleeClients`. We also apply conditional under-relaxation $Z_RELAX_ALPHA = 0.5$ in `updateTaskLayerClientsThinks` when `callerFansOut` is true, so that single-callee chains keep their un-damped fast path whilst fan-out callers get damping to limit the oscillation from multiple simultaneously-updated channels.

3.5.3 Rejected alternatives

We also had a look at a couple of alternatives for the final design, however neither was adopted. We attempted a Little's-law warm-start with bottleneck bounds. Whilst this approach saved one iteration, it was at the cost of two new formulas, gating heuristics, and warm-start machinery, so was judged as not worth the conceptual overhead. Another approach was `relax_omega` adaptive damping. The goal was to detect oscillation via sign-change, however for fan-out workloads, the false-saturation locks in early, meaning the adaptive damping only arrives after a bad fixed point is set. A considered approach was to gate fixed- α damping on a predicate that identifies fan-out callers statically (`LqnGraph.callerFansOut`), so damping is on from sweep 1 for the topology that needs it. This is the C-1 novelty candidate flagged in the related-work survey.

3.5.4 Refining multi-class × multi-entry interactions

Once the fan-out fix is in, we look to finalise any remaining bugs. Those found were per-class apportionment of pushed-back demand (had been hard-coded into a single class), per-call-vs-per-

visit semantics for multi-entry callers (calls dispatched by a bound activity to multiple entries should be averaged not summed across entries, as each visit goes to one entry), summing per-class throughput and queue length in coupling propagation (had been reading class-0 only), and a leaf-vs-intermediate `cMean` distinction for sojourn-time multiplication. A results-side regression also surfaced, where a single `entryQlenFraction` map was used for both QLen and Util, as Util needs host-demand-only weighting ($X \cdot D / c$) while QLen needs full per-call weighting.

3.5.5 What fan-out handling provides

Large fan-out models now converge to the correct result in roughly 24 outer iterations rather than locking in a false fixed point at 3 iterations. The multi-class $\times$ multi-entry refinement clears all regressions on the test corpus.

The structural lesson from the fan-out implementation is that the iteration’s dynamical structure matters, not just per-sweep math. Gating fixed- α damping on a structural predicate (`callerFansOut`) means damping is on from sweep 1, avoiding the otherwise required detection iterations. However, the trade-off is that the solver cannot adapt to model classes we have not yet considered.

3.6 Activity-graph awareness

When the solver was fed LQN models with richer activity-graph features, its bag-of-activities reading of each entry returned incorrect results. The pre-activity-graph solver treated every entry as if its bound activity was the whole entry’s per-call cost and treats every task’s activities as a flat unordered bag. This therefore doesn’t account for the branch probabilities, loop counts, parallelism and the synchronous-wait boundary. We solve this with a per-activity visit-weight DAG walk over `ActivityPrecedence`, an AND-fork correction that replaces the branch-sum with the inclusion-exclusion $E[\max]$ of branch demands and a phase1-phase2 split that narrows the caller-perceived demand at `REPLY`. Figure 3.4 shows an example activity graph that exercises these features.

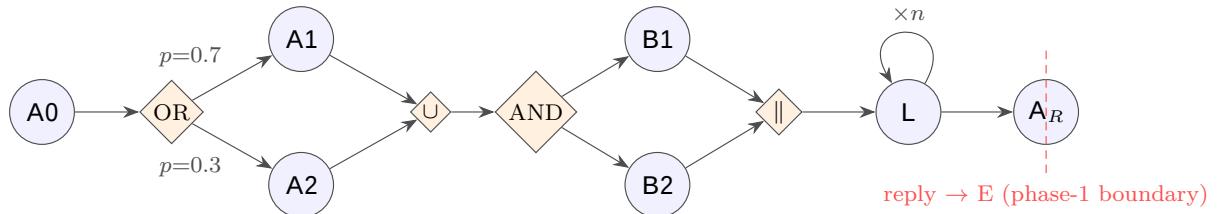


Figure 3.4: An example activity graph model.

3.6.1 Failure:

Six biases follow from the pre-activity-graph assumptions:

- `LqnGraph.hostDemandOfBoundActivity(Entry)` reads only the bound activity’s host demand, so **multi-activity entries whose sequence runs past the bound activity are under-counted**.
- `LqnGraph.perVisitCallsFromCallerToTarget`, `SolverLNSimple.computeSiblingCalleeBlocking`, and `EnsembleInitialiser.demandFromActivityToTargets` sum call Mean over `task.getActivities()` as a flat bag, meaning **OR-fork branch probabilities are dropped**.
- The same flat-bag read ignores how many times a loop body fires, meaning **loop counts are dropped**.

- AND-fork / AND-join is entirely absent from the pre-activity-graph design, treating the calls as independent. Therefore, **parallel branches are summed instead of joined**.
- A `repliesTo(E)` call ends the caller's synchronous wait at the reply, but the post-reply phase-2 work is considered in the caller's per-call demand, meaning **a phase-1 REPLY inflates the caller-perceived demand**.
- The same folding applies when the reply sits on the bound activity rather than mid-sequence, collapsing phase 1 to that single activity, meaning **a bound-activity REPLY inflates the caller-perceived demand**.

3.6.2 Response:

We address these issues with a DAG walk over the entry's activity graph. Firstly, we replace the bag read with a per-activity visit weight and apply it at every call-accounting site. We then add an AND-fork join-delay correction, applied to the reported metrics after the iteration rather than inside its coupling loop. Finally, we split each demand into a phase-1 and a phase-2 part, so the caller is charged only for the work up to its reply while the processor still sees the full demand.

Visit-weight DAG walk

We replace the bag-of-activities read with a per-activity *visit weight*, derived from a DAG walk over the entry's `ActivityPrecedence` chain. We compute these weights in `LqnGraph.computeActivityVisitWeights(Task)`, summing per-entry walks for multi-entry tasks, and `computeWeightsFromBound(Task, Activity)`, a fixed-point iteration that starts at the bound activity and propagates through precedences via `applyPrecedence`.

The precedence handling is as follows: `PRE_SEQ` inherits, `PRE_AND` collapses by `max`, `PRE_OR` sums, `POST_SEQ` takes a single forward edge, `POST_AND` each successor inherits the fan-in, `POST_OR` each successor inherits `fanIn × prob`, and `POST_LOOP` scales by loop count.

`hostDemandOfEntry` walks the entry's DAG and sums `visitWeight × hostDemand` over reachable activities, giving the full demand the entry contributes per call. We follow the same idea for all other call-accounting sites that read activities as a bag, including `perVisitCallsFromCallerToTarget`, the multi-class per-class loop in `propagateTaskLayerCoupling`, and `OutputTableBuilder.appendActivityRows` (where the weight instead scales reported per-activity throughput).

AND-fork E[max] correction

We then look to handle fork-join structures. We initially assumed the caller-perceived join delay was the deterministic max of branch demands, however we eventually recognised the pattern as the inclusion-exclusion identity for the expected maximum of independent exponentials [5]. For two parallel branches with exponential demands of means D_1, D_2 , the expected time for *both* to finish is the inclusion-exclusion expected maximum $E[\max] = D_1 + D_2 - \frac{1}{1/D_1 + 1/D_2}$. This extends to the general case of $E[\max] = \sum_{\emptyset \neq S \subseteq \{1, \dots, K\}} (-1)^{|S|+1} \left(\sum_{i \in S} \frac{1}{D_i} \right)^{-1}$ for K branches.

The implementation adds `taskHasAndFork(model, taskName)` (predicate gating the correction), `findMatchingAndJoin(precs, branchStarters)` (bijective forward-reachability), `firstReachableJoinPre(start, joinPreActs, precs)` (`POST_SEQ` depth-first search), `collectBranchDemand(task, branchStart, joinPreActs, weights)` (sums `weight × hostDemand` along one branch), and `andForkMaxCorrection(task, weights)` (sums $(E[\max] - \sum \text{branchDemand})$ over matched fork/join pairs), and `expectedMaxOfExponentials(means)` (inclusion-exclusion over $2^K - 1$ subsets). The demand functions now have to utilise this where needed, with `hostDemandOfEntry` now collapsing parallel branches via $E[\max]$ whilst `processorDemandOfEntry` reports the full SUM.

REPLY phase-1 / phase-2 split

When an activity calls `repliesTo(E)` mid-sequence, the caller’s synchronous wait ends, and subsequent activities are post-reply background work that the processor still pays for but the caller does not directly perceive. Considering these as one inflates the caller’s per-call demand by the phase-2 body. We add `phase1ActivitiesFromBound(entry, bound)` (a forward DAG walk from the bound activity along `POST_SEQ`-like edges that stops at any reply activity) and `hasPhase2Background(entry)` (full reachable set minus phase 1).

The demand functions are now split again. For example, `hostDemandOfEntryImpl` is re-parametrised over `sumBranches` and `phase1Only` flags. We now have `hostDemandOfEntry` (phase-1 only $E[\max]$ join modelling the caller-perceived synchronous wait), `processorDemandOfEntry` (full DAG sum modelling processor occupancy), `andForkCollapsedDemandOfEntry` (full DAG $E[\max]$ join isolating the AND-fork delta from the phase split) and `phase1SumDemandOfEntry` (phase-1 raw sum).

A fresh caller can arrive while phase-2 is still busy. `phase1AdjustedResponseTime` reports the caller-perceived response time $R = S_1 + \text{prOt} \cdot S_2$ (always paying the phase-1 residence S_1 , plus the phase-2 residence S_2 when, with probability `prOt`, this case occurs). We get `prOt` from a small continuous-time Markov chain over the server’s three states (idle, phase-1, phase-2). Solving it under Poisson arrivals gives $\text{prOt} = \lambda\mu_1 / (\lambda\mu_2 + \mu_1\mu_2 + \lambda\mu_1)$, however this assumes an unbounded arrival stream, which over-states the arrival rate near saturation. We replace λ with an effective rate $\lambda_{\text{eff}} = \lambda / (1 + \rho)$, where $\rho = \lambda D / c$ is the host utilisation.

3.6.3 Rejected alternatives

To keep all host layers in consistent units, an early version threaded explicit per-cycle / per-call basis tracking through three places: the writeback, the residence-time division, and the Q scaling. Despite correctness, it was removed in favour of a one-line change to the self-demand fallback in `setAggregateHostDemand`. That one line puts every host layer into per-cycle units at the source, making all the downstream tracking redundant. Separately, `POST_LOOP` with no end activity was first handled by writing only `loopCount` to the post side, but the resulting fractional self-loop did not converge fast enough. We now simply scale the loop body’s existing demand by `loopCount` in place, so the repeated work is folded in directly, providing better convergence and a cleaner implementation.

The interesting question was then where to apply the corrected $E[\max]$ join. Applying it inside the iteration (through `writeHostResponseToTaskLayerServer` on each sweep) failed, but keeping the iteration on a plain SUM and applying $E[\max]$ just the once after convergence did not. The reason is that $E[\max]$ is a non-additive function of the branch residence times, whereas the iteration’s demand accounting is purely additive, therefore feeding the max back in each sweep makes downstream layers treat it as an ordinary summable demand, pushing estimates through nonlinearity and converging to a wrong fixed point.

One case left out of scope for our simple solver is an AND-fork whose sibling branches run on the same finite-server host. The branches contend for the host’s servers and overlap in time, so the plain $E[\max]$ join over-estimates the join delay. Modelling that overlap correctly needs CCD compensation [25], requiring a large amount of invasive machinery for one narrow case.

3.6.4 What activity-graph awareness provides

Activity-graph awareness lets the solver express five constructs the bound-activity model could not: multi-activity entries that sequence past the bound activity, probabilistic OR-fork branching, `POST_LOOP` wrapping a call-issuing body, AND-fork / AND-join with caller-perceived $E[\max]$ join delay on INF-hosted branches and mid-sequence REPLY with post-reply phase-2 background under the three-state overtake correction. The one gap left is where a finite-server PS host under AND-fork contention would need the CCD overlap compensation we chose not to build.

The whole demand calculation is derived naturally from a single DAG traversal, where `host`

`DemandOfEntryImpl(sumBranches, phase1Only)` produces the four quantities the iteration, the result correction, the processor-occupancy term and the displayed caller-perceived response time need. The $E[\max]$ correction came the other way round, understanding the numerical differences to the reference solvers, and then recognising them as the Heidelberg-*Trivedi* identity. We also learn that the fixed-point solution can be affected if we add non-linearity to the coupling loop, hence keeping some corrections to the results only.

3.7 Optimising the solver

After reaching a point where the solver is correct on all cases of models we consider, excluding niche edge cases and complex corrections, we look to optimise the solver’s performance. While `SolverLN` and `LQNS` must keep their convergence machinery general enough for every model they might ever be asked to solve, we know exactly the types of models `SolverLNSimple` targets. This freedom already lets it shed iteration cost and per-query cost, but we can go further by specifically optimising for the set of models we have.

This section is performance-motivated, optimising the convergence behaviour of the outer fixed-point loop and the per-sweep cost of the topological lookups the coupling performs. We look at a limit-cycle detector in the outer loop to prevent oscillations, memoisation of the invariant topological queries to avoid redundant computations and a revision of the convergence rule and damping features. We also discuss why classical sequence acceleration does not provide benefit for our solver. The de-generalisation `SolverLNSimple` utilises, because it only ever mutates the per-class Z/D values and not the topology those queries read, means its benefit is reduced allocation rather than measurable wall-clock. A warm-start of the coupling estimates was implemented, measured, and dropped, whilst an experiment on sweep order was also tried, and despite showing minimal performance improvements we switch to a pure elevator approach that allows comparison of iterations in the Evaluation chapter to be more like-for-like. The result is a 34.7% reduction in outer iterations across the corpus ($568 \rightarrow 371$), with a runtime decrease of 54.9% ($1739\text{ms} \rightarrow 784\text{ms}$).

3.7.1 Where the cost sits

The correct solver carries two kinds of slack, each with concrete evidence.

Structural iteration overhead. Despite all models expected to show correctness doing so, we found a chain + fanout model that hits the `maxIter = 100` of our outer loop. From its 14th iteration onward, the maximum throughput delta stays consistent between iterations, with state updates alternating $A \rightarrow B \rightarrow A \rightarrow B$ in a perfect two-cycle. The two-cycle is structural as our LQN coupling’s feedback loop has a caller’s `Clients think` depends on its callee’s response time, which in turn depends on the caller’s `think`. When a caller fans out into a near-saturated callee, the loop supports two locally-stable states. Another form of overhead is the convergence tolerance $\Delta < 5e-3$, something we haven’t considered changing since its inception, being about five times tighter than the `rtol = 5e-2` the harness actually judges against, causing perhaps unnecessary tightening. Furthermore, the per-iteration update is damped at a set $\alpha = 0.5$ on both the fan-out caller `think-time` and the host-response write into INF tasks, halving the step toward the fixed point, on all fixtures, despite many not needing the damping.

General-purpose query cost. A JDK Flight Recorder (JFR) profile of the solver places `LqnGraph.findEntry` (~54 CPU samples) and `LqnGraph.findCallerTask` (~26 samples) at the top of the samples, as seen in Figure 3.5. Both are linear scans over the `LayeredNetwork` topology, called from inside the coupling stages on every sweep of every layer. The topology, however, is invariant during a solve, yet we still force the scans to recompute every sweep. That is the kind of redundancy that a stateless, model-agnostic helper must tolerate, however our specialised solver, knowing only the Z/D values change while the topology is fixed, can exploit it.

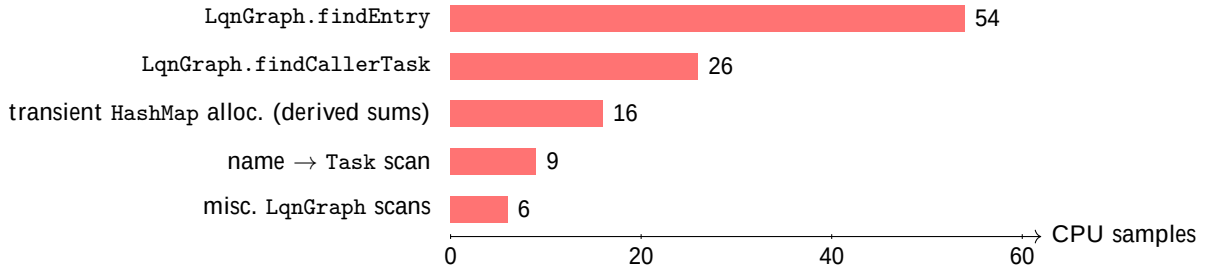


Figure 3.5: Top CPU samples in the pre-optimisation JFR profile.

3.7.2 Response

The points we make are ordered as structural features first and parameter tuning last, as whilst the features are independent of each other, the parameter choices are only safe on the state they establish, so the tuning comes once the algorithm is settled.

Limit-cycle detection

The two-cycle is fixed not by adding damping (an α sweep showed the iteration count on this fixture is α -invariant, muting the amplitude without breaking the cycle) or by extrapolating the alternating values (the classical acceleration approach that recognises non-convergence). Instead, the right response is to detect the pattern directly and exit with the answer the solver already has.

Inside `iterateCoupledMva` we track `prevMaxDelta` and, after the existing $\Delta < \text{tol}$ check, test $|\Delta_k - \Delta_{k-1}| / \max(\Delta_k, \epsilon) < \text{CYCLE_REL}$ for `CYCLE_WINDOW` consecutive iterations, gated by $\Delta_k < \text{CYCLE_DELTA_CAP}$ to keep the detector silent through the early contraction phase. The chosen constants are `WINDOW = 1`, `REL = 1e-3`, `DELTA_CAP = 10`. This detection not only catches two-cycles where $\Delta_k = \Delta_{k-1}$, but also the smooth contraction tail of chain fixtures, where the effective contraction ratio approaches 1. The 100 iteration case now drops to 6, and chain fixtures each save a single tail iteration.

Because of these two cases, the detector returns differently depending on the shape of the stall. For a genuine two-cycle pattern, the iteration alternates across the fixed point, so their average is the better estimate. For slowly-converging chain tails, the iterations approach monotonically from one side of the fixed point, so picking the most recent result is better. We distinguish the two by checking if a sign flips between consecutive steps, $(X_k - X_{k-1})(X_{k-1} - X_{k-2}) < 0$, signifying oscillation.

Memoising the invariant topological queries

The profiled hotspots are removed by computing each invariant query once and reusing it, and two patterns emerge.

Single-field reads get delegated to `LayeredNetworkStruct`. For example, `findCallerTask`, the documented hotspot, is replaced by an `int[] callerOfT` in initialisation, derived from `model.getStruct().issynccaller`, instead of a triple-nested walk (Task → Activity → Call Destination) over the live `LayeredNetwork` object graph. A single-field lookup does better as a struct read than as a cached result, because it carries no first-call penalty and no map state.

Everything else is *memoised*. A `WeakHashMap<LayeredNetwork, ModelCache>` with a volatile fast path (`lastModel/lastCache`), so `cacheFor()` is a pointer-identity check rather than a synchronised map lookup, holds both the name primitives (e.g. `findTask`) and the derived quantities (e.g. `computeLocalDemand`), with the same pattern applied to `MvaInputs` and to the per-submodel `Network` caches on `SolverLNSimple`. Whilst `LayeredNetworkStruct` holds similar quantities, some values may disagree with the semantics of our solver. For example, our AND-fork handling takes the max of weights whilst the struct `graph` sums branch-tip weights,

so we memoise the solver’s semantics directly rather than trying to re-use the struct’s values.

The iteration count is unchanged as the solver runs the same algorithm and reaches the same fixed point, however each redundant topology scan that we eliminate prevents a short-lived `HashMap` from being built, saving roughly 10–15k of these short-lived allocations per corpus run. Because the JVM’s garbage collector has to do work in proportion to how many such objects are created and thrown away, fewer allocations means less collector work, however this was not directly measurable as the collector was never the bottleneck. What the change does surface is a profile-driven $2\times$ performance win across the corpus ($1010 \rightarrow 497$ ms) because the repeated name lookups were a genuine hotspot.

Asymmetric revision of the convergence rule

`SolverLNSimple` is primarily positioned as an *efficient* solver and not a general and perfect-accuracy one, whereas `SolverLN` and `LQNS` are. Considering this, our original convergence and damping criteria is overspecified.

The damping in the solver was not arbitrary, and was targeted to callers invoking more than one distinct callee, such that the new Z is under-relaxed because MVA’s response slope in Z is steep near saturation, so plain fixed-point iteration alternates between saturated and unsaturated states, and that INF tasks on multi-server PS processors damp the D write since `R_proc` fluctuates between bare-demand and queueing-tail values across sweeps. Both phenomena are real, however the cycle handling is now done structurally by the two-cycle detection, meaning the fluctuation concern that motivates damping is no longer an issue.

Table 3.1 reports the joint (α_Z, tol) sweep with $\alpha_D = 1.0$ fixed. The result exposes an asymmetry. **For D**, dropping the damping from 0.5 to 1.0 uniformly saves iterations with no correctness regressions across the corpus. **For Z**, the opposite holds, such that increasing α_Z from 0.5 toward 1.0 inflates the corpus iteration count. At $(\alpha_Z = 1.0, \text{tol} = 1.5\text{e-}2)$, the corpus total is 456, while at $(\alpha_Z = 0.5, \text{tol} = 1.5\text{e-}2)$ it is 371, a penalty of 85 outer iterations. The cause of this is the saturated fan-out fixtures, where $\alpha_Z = 1.0$ drives them towards the iteration limit as the caller-callee blocking feedback loop in fan-out topologies supports two locally-stable states at unit step size that Z damping suppresses.

$\alpha_Z \setminus \text{tol}$	5e-3	1e-2	1.5e-2	5e-2
0.5 (chosen)	412	388	371 ←	325
1.0	484	463	456	428

Table 3.1: Joint (α_Z, tol) sweep with $\alpha_D = 1.0$ fixed (78-fixture corpus, outer iterations; 0 correctness regressions throughout). $\alpha_Z = 0.5$ dominates at every tolerance; `tol = 1.5e-2` is the chosen column.

The tolerance is loosened from `tol = 5e-3` to `tol = 1.5e-2` on convergence-margin grounds, as our tolerance was much stricter than our test pass acceptance for the corpus. The pass/fail corpus boundary lies near `tol  $\approx$  2e-1`, whilst our chosen `tol = 1.5e-2` is $3.3\times$ smaller than the LN `rtol = 5e-2`. At the chosen $\alpha_Z = 0.5$, $\alpha_D = 1.0$, the tolerance loosening saves a further 41 iterations ($412 \rightarrow 371$). Dropping the D-side damping is the larger absolute saving ($484 \rightarrow 412$ at `tol = 5e-3`), as the damped $\alpha_D = 0.5$ update had been accepting an intermediate value once Δ fell below tolerance rather than landing on the fixed point, whereas the un-damped step lands closer. Z retains its damping, with $\alpha_Z = 0.5$ as the corpus-optimal value that keeps the saturated fan-out fixtures convergent.

3.7.3 Rejected alternatives

The optimisation work further tested three families which are worth recording and despite their failures, justify the optimisations that stuck.

Closed-form warm-start

A warm-start writes an initial Clients-think value into each T: layer before the outer loop, using the unsaturated branch of `computeCallerClientsThink` ($z = \max(\text{refChainThink}, \text{directThink}) + \text{siblingBlocking}$). The idea is that seeding each layer with a closed-form estimate of its converged think time places the initial guess closer to the fixed point, so the outer loop has less ground to cover. While this saved a few outer iterations across the slow fan-out fixtures, the idea was dropped because it provided no execution time gain on the corpus, as the first iteration of work was simply moved to outside the convergence loop. Therefore, it required extra machinery with no real benefit.

Classical sequence acceleration is the wrong lever

Aitken’s Δ^2 extrapolation, Anderson acceleration, Nesterov momentum, and Broyden quasi-Newton iteration [26] were each implemented, and each was rejected. The idea is that Aitken extrapolates three iterations of a linearly-converging sequence to its fixed point, Anderson generalises this to a least-squares fit over a window of recent residuals, Nesterov extrapolates ahead by a momentum term before each step and Broyden maintains a rolling inverse-Jacobian estimate and takes a Newton-like step. However, all four assume that the iteration is locally smooth, that the contraction is linear-rate at a moderately slow ratio, or that there are enough iterations to fill the acceleration window. The LQN decomposition iteration has none of these properties. The outer contraction is already strong and iteration counts stay small (typically 3–10) for most models so there is no slow linear tail to extrapolate, and the acceleration windows never fill. Therefore the smoothing methods are actively harmful, inflating iterations by 24–65% as momentum amplifies oscillations, and giving eight correctness regressions as Jacobian estimate jumps between two states of the cycle.

3.7.4 Sweep order

The default *bounce-sweep* (a $2N - 1$ up-and-down pass per outer iteration) was re-examined against forward, reverse, double-forward and elevator (which `SolverLN` and `LQNS` use) patterns. *Forward* is simply a $1, \dots, N$ sweep per iteration, *reverse* is $N, \dots, 1$, *double-forward* is $1, \dots, N, 1, \dots, N$ to attempt a similar workload per iteration to bounce-sweep, and *elevator* alternates between forward and reverse each iteration. Runtime results measure just the solve phase, excluding noise from other parts of the overall run. Table 3.2 reports the comparison.

Order	iterations	regressions	runtime
bounce-sweep	371	0	1033 ms
forward	608	2	1089 ms
reverse	413	1	881 ms
double-forward	373	1	1250 ms
elevator	539	0	976 ms

Table 3.2: Sweep-order comparison experiment over the corpus.

Our hypothesis that these patterns do different amounts of per-layer MVA work per outer iteration, so their iteration counts are not directly comparable holds. Bounce and double-forward run $2N - 1$ per-layer MVA solves in a single bidirectional pass whereas the single-direction forward, reverse and elevator orders run N solves per iteration. We therefore look at runtime as a primary metric to compare the patterns, as a direct 2-to-1 iteration ratio does not hold.

The observed gap is instead $1.45\times$ in iteration count, yet elevator cuts the solve-phase runtime by 5.5% despite this. The elevator converges faster per iteration ($1.45 < 2$) as it propagates fresh results up and down the call graph. It also achieves a better runtime performance ($976 < 1033$) because of this, with the total solve work being roughly $539N$ against $371(2N - 1)$, however this is somewhat offset by the fixed per-iteration costs.

An interesting case is the reverse pattern, that reports the least runtime, however it provides

one correctness regression. The performance benefits were because of the sweep visiting the leaf callees before their callers, propagating fresh results quickly within the same pass, matching the dependency direction of the decomposition. However, when we inspected it further we found it was unsound, as under fan-in saturation the sweep order becomes load-bearing, where reverse solves the shared callee first with every caller throughput stale and settles into a self-consistent-but-wrong fixed point. The two-direction passes enforce per-iteration consistency, ensuring correct fixed-point lock-in.

We choose to migrate to the elevator pattern, however mainly for convenience of the iteration count comparison in the Evaluation chapter, rather than for the marginal runtime improvement.

3.7.5 What optimising the specialised solver provides

Before the sweep order change, the optimised solver converged on the 78-fixture corpus in 371 outer iterations against the unoptimised 568, a 34.7% reduction, and in 784 ms of execution time against 1739 ms, a 54.9% reduction.

The iteration saving comes from the limit-cycle detection (~ 84 iterations, although mainly on one fixture) and dropping D damping and loosening the tolerance (113 iterations), resulting in a $568 \rightarrow 371$ improvement.

The name memoisation was a profile-driven $2\times$ win relative to workload, with the de-generalisation specialisations contributing robust JVM allocation relief. These numbers are a separate contribution from, and do not revise, the diagnosis of `SolverLN`'s structural overhead, instead targeting its generalisability as a performance constraint.

Three structural lessons are as follows. *Workload-relativity*: an optimisation stack is a coupled system where each layer's value depends on what else has been done, meaning a profile-driven win can be silently consumed by a later change and re-measurement after each step is mandatory. *Joint parameter sweeps catch what single-knob sweeps miss*: a marginal sweep rejected the entire convergence-rule move, and only the joint (α_Z, tol) grid revealed the asymmetry between Z and D damping. *Specialisation at its honest level*: the simple solver is free to specialise precisely because it is not a general-purpose meta-solver. We see this in the de-generalisation specialisations enabled by the topology-invariance and the convergence-level changes motivated by not requiring perfect accuracy.

3.8 Closing remarks

The chapter has presented `SolverLNSimple` as a set of capabilities, starting from a minimal LQN solver and looking at the features that extend it. Each feature responded directly to a specific model failure that a design without it could not handle, and the machinery it adds is what its specific failure demanded. The `LqnGraph` abstraction emerged from the graph-derived coupling work, accumulated through multi-class support and fan-out handling, and became the class the codebase organises around. The design choice of the project was to always pick the solution that requires the least reworking or extra machinery, to prevent the code from becoming overly complex and falling into the same trap as the mature solvers.

However we have not yet measured the results. The Evaluation chapter reports the iteration counts and accuracy data on closed-LQN fixtures spanning single-class chains, multi-class shared-callee topologies, deep IS topologies at large populations, fan-out callers with sibling callees, multi-class-multi-entry interactions and activity graph uses. We compare against `SolverLN` [15] and LQNS V5 [6, 21]. The structural claim the chapter has made, that the simple solver omits machinery it does not need as a design, is what makes the iteration-count comparison meaningful when the evaluation reports it. The contribution is the construction and optimisation itself, with its characterisation that the bottom-up build hits fewer iterations than both comparators, and the per-machinery decomposition in the evaluation showing that most of the iteration-count difference with `SolverLN` is structural, but not universally unnecessary.

Chapter 4

Evaluation

4.1 Scope and Hypotheses

This chapter measures `SolverLNSimple` against the two reference solvers it was built to stand beside, `SolverLN` and `LQNS`, on a corpus of closed LQN models. It is organised around five measurement goals, sharpened into seven falsifiable hypotheses (H1–H7) and bounded by three explicitly conceded limits.

The **spine claim** the chapter characterises is that the difference between `SolverLN` and `SolverLNSimple` is real, but can be decomposed across both iteration-count and runtime. On *iteration-count*, approximately three-quarters of the gap between the solvers is attributable to `SolverLN`’s convergence control, most notably its `iter_min` floor, which masks the true fixed-point convergence rate, while the remaining quarter is algorithmic difference in how the two solvers couple and damp their layers. On *runtime*, `SolverLN`’s interlock correction imposes per-iteration cost that grows with fan-out breadth while having no effect on iteration count or accuracy on this corpus, as it is not a topology we have considered in this project. The price `SolverLNSimple` pays for omitting the two machineries is a small accuracy cost on a minority of fixtures, each of which still lands within the tolerance bounds of a reference.

Five measurement goals: The corpus we measure against is built top-down from five goals, each answered by one section of the chapter. The first goal is *correctness*: does `SolverLNSimple` reproduce a reference’s converged values on the models it claims to handle (Section 4.3)? The second is *iteration count*: how many fewer outer iterations does it need than `SolverLN` and `LQNS` (Section 4.4)? The third is *runtime*: how much wall-clock faster is it, and is the per-iteration cost competitive (Section 4.5)? The fourth is *attribution*: which specific `SolverLN` machinery produces the iteration-count and runtime gaps (Section 4.6)? The fifth is *boundary*: where do the simple solver’s approximations break, and is that breakage confined to cases outside the claimed scope (Section 4.7)? Partition A of the corpus is the positive-evidence set for goals 1–3, Partition B the scale axis for goals 2–3, and Partition C the boundary set for goal 5.

The seven hypotheses:

H1: Correctness on the positive-evidence partitions: *Confirmed by:* On every fixture the build claims to handle (all of Partition A, all of Partition B), `SolverLNSimple` matches a reference within 5% on every metric, each being categorised to `match-ln` or `match-lqns-only`. *Falsified by:* any `match-neither` fixture in Partition A or B. *Evidence:* Section 4.3, Table 4.1.

H2: Iteration-count reduction. *Confirmed by:* `SolverLNSimple` converging in substantially fewer outer iterations than `SolverLN` on every fixture, and fewer than `LQNS` in aggregate. *Falsified by:* any fixture where `Sim` $\geq$ `SolverLN`, or a corpus total above `LQNS`. *Evidence:* Section 4.4, Table 4.2.

H3: Runtime. *Confirmed by:* `SolverLNSimple` producing a dramatically faster than `SolverLN` and being competitive with `LQNS`. *Falsified by:* a corpus total above `SolverLN`, or more than an order of magnitude off `LQNS`. *Evidence:* Section 4.5, Table 4.3.

H4: The interlock correction is a pure runtime-surface cost. *Confirmed by:* Dis-

abling `SolverLN`’s interlock correction providing no change in fixture iteration count or accuracy classification, but removes per-iteration runtime. *Falsified by:* any iteration-count or accuracy change, or no runtime decrease. *Evidence:* Section 4.6.1, Table 4.6.

H5: Convergence control dominates the iteration-count gap. *Confirmed by:* Disabling `SolverLN`’s convergence control closing most of the iteration-count gap to `SolverLNSimple` *Falsified by:* a closure below 60%, which would locate the gap elsewhere. *Evidence:* Section 4.6.2, Table 4.8.

H6: Convergence control carries a small, concentrated accuracy cost. *Confirmed by:* Disabling convergence control moving a minority of `SolverLN`’s own outputs off the fixed point, with limited drift. *Falsified by:* a diffuse drift across the whole corpus, or none at all. *Evidence:* Section 4.6.2, Table 4.9.

H7: A residual algorithmic difference remains. *Confirmed by:* Even with convergence control disabled, `SolverLN` still uses more iterations than `SolverLNSimple`. *Falsified by:* reduced `SolverLN` iteration count ≤ 539 . *Evidence:* Section 4.6.3, Figure 4.6.

4.2 Instrumentation

For each model in our test corpus, we run `SolverLNSimple`, `SolverLN`, and `SolverLQNS`. The test is marked as passing if our solver matches either solver within 5% tolerance on all metrics, as sometimes even the mature solver outputs differ. We also use performance counters. Iteration counts for each solver are retrieved (`iters[0]`, `SolverLN.maxitererr.size()` and `SolverLQNS.result.iter` respectively), along with a clock execution time via two calls to `System.currentTimeMillis()`, allowing us to compare performance across solvers, however the iteration counters cannot be directly compared.

A `@BeforeAll` warmup runs a test fixture three times, with data capture suspended, sufficient to JIT-compile the core MVA kernels and solver loops. We also run each solve when performance testing three times and take the median value to mitigate any sources of contention on the device.

4.2.1 Iteration counters

Whilst the three counters provide a good insight into the iteration behaviour, they are not perfectly comparable. `SolverLN`’s reported count includes zero-padding prepended to `maxitererr`, plus a hard-reset verification pass after the main convergence loop terminates. The reported count is therefore `actual_iters + 2` at minimum, biasing the comparison slightly away from `SolverLN`. Beyond this offset the per-iteration work is matched: `SolverLNSimple` sweeps the layers in a single direction per outer iteration (N per-layer MVA solves, using the elevator order that alternates a forward and a reverse pass) exactly as `SolverLN` and `LQNS` do. The earlier bounce-sweep $2N - 1$ -versus- N asymmetry therefore no longer arises, so no per-iteration-work deflator is required and we report the raw iteration counts as a like-for-like comparison.

4.2.2 Test corpus

The 78-fixture `LayeredNetwork` evaluation suite is constructed top-down from the five measurement goals stated at the start of this chapter, separate from the collected corpus accumulated through the build history, which served as a regression net in development. The separation is desired as the corpus that drove construction cannot independently grade it, becoming victim to survivorship bias.

Partition A (43 fixtures) is the positive-evidence set spanning the spine’s claimed scope, containing single-class chains, multi-class shared-callee under three asymmetry patterns, multi-server processors, activity-graph capabilities and topology configurations approximating realistic multi-tier systems.

Partition B (10 fixtures) targets execution time scaling for performance experiments. It consists of linear chains and fan-out trees with increasing task counts. Population is held at $N=30$ across the partition as the original $N=50$ caused `SolverLN` to timeout.

Partition C (25 fixtures) is the boundary set, deliberately constructed near regimes where the spine claim’s approximations are expected to fail rather than to confirm them. Fan-out models target the topologies most likely to exhibit inter-layer oscillation in coupling, INF-scheduled callee configurations exercise the bare-demand heuristic, multi-class multi-entry configurations probe the interaction surface and boundary cases at near-saturation ($\rho \approx 1$), the structural-predicate threshold ($E[\max]$) and near-trivial load (deep all-Exp / multi-call chains) characterise where the simple solver’s approximations transition.

4.3 Correctness

The iteration-count and runtime claims that follow are only meaningful if the simple solver’s outputs are numerically faithful where it claims them to be. Correctness is confirmed by **SolverLNSimple** landing within the project’s 5% tolerance ($\text{atol} = 1\text{e-}3$, $\text{rtol} = 5\text{e-}2$) against either reference solver, on every metric. The two-reference test is necessary as the references themselves disagree on some topologies. Table 4.1 maps where this holds, broken down by partition.

Partition	N	match-ln	match-lqns-only	match-neither
A1 (single-class chains)	5	5	–	–
A2 (multi-class shared callee)	12	12	–	–
A3 (multi-server PS)	9	9	–	–
A4 (multi-tier topology)	10	10	–	–
A5 (activity graphs)	7	7	–	–
B (scale axis)	10	5	5	–
C1 (fan-out callers)	6	–	6	–
C2 (INF-task hosts)	3	3	–	–
C3 (multi-class $\times$ multi-entry)	3	3	–	–
C4 (boundary cases)	3	3	–	–
C5 (saturated fan-out)	6	–	3	3
C6 (AND-fork join boundary)	2	–	–	2
C7 (deep chains)	2	–	–	2
TOTAL	78	57	14	7

Table 4.1: Where correctness holds, by partition.

The failures are structural and expected. Across the 78-fixture corpus, 71 of the tests pass (**57 match-ln**, **14 match-lqns-only**), whilst only **7 match-neither** (Table 4.1). We see all of Partition A (correctness) and all of Partition B (scale axis) pass, whilst just 7/25 of the Partition C (boundary/failures) fixtures match-neither. These fixtures were *constructed* to probe the solver, with each naming a structural limit we are aware of. We classify this as a success, as there is no failure that lands in the positive-evidence partitions (expected pass).

Partition A: the positive-evidence set. Partition A’s 43 fixtures span the spine’s claimed scope with single-class chains (A1), multi-class shared-callee patterns (A2), multi-server PS hosts (A3), multi-tier and mixed topologies approximating realistic systems (A4), and activity-graph precedence (A5). Each of the 43 tests match **SolverLN**’s converged values to within tolerance on every metric. We look at the residual between the **SolverLNSimple** and **SolverLN** results, and see that our results are safe. First, the single-class, multi-server, and activity-graph fixtures (A1, A3, A5 OR/seq) clear the $\text{rtol} = 5\text{e-}2$ bar by three to seven orders of magnitude, showing exact reproductions of **SolverLN**’s converged values. Second, where the residual does grow, it grows along a single, predictable case (as seen by the clustering in Figure 4.1): the multi-class and multi-entry fixtures. Here **SolverLN**’s per-iteration class-switching machinery does its load-bearing accuracy work, with the error climbing monotonically with that structural complexity (A2 at $2.6\text{e-}03$, A5 reply/DAG at $1.3\text{e-}02$, A4 at $4.76\text{e-}02$).

Why the seven Partition C failures are expected. All seven **match-neither** fixtures sit in three of the boundary subpartitions (C5, C6, C7), seen in Table 4.1. They are constructed

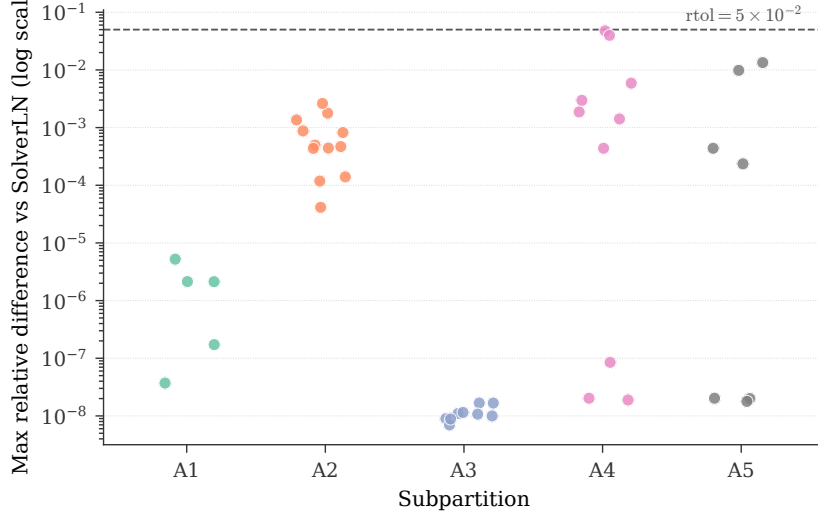


Figure 4.1: Per-fixture maximum relative difference between `SolverLNSimple` and `SolverLN` on Partition A.

specifically to push the simple solver’s closed-form kernels past the regimes they are derived for. The three saturated fan-out fixtures (C5) drive per-server utilisation to $\rho \approx 0.99$, where the simple solver’s closed-PFQN M/M/c queueing tail and `SolverLN`’s M/M/ ∞ -like $R \approx D$ host substitution diverge. The two AND-fork fixtures (C6) drive the closed-form $E[\max]$ join correction past its independent-server assumption. The two deep all-Exp chain fixtures (C7) converge quickly but to a steady state differing from both references, locating where the bare-demand closed-form kernel transitions under tight coupling.

4.4 Iteration-count comparison

The chapter’s headline claim is that `SolverLNSimple` converges in substantially fewer outer iterations than the two comparator solvers across the corpus. This section reports those counts, inspects them by model partition for clarity, and acknowledges the one comparison bias towards `SolverLNSimple`’s count (`SolverLN`’s +2 hard-reset offset). With all three solvers sweeping in an elevator pattern, the counts are otherwise like-for-like.

Total iteration counts.

Across the 78-fixture corpus, `SolverLNSimple` uses 539 outer iterations in total, `SolverLN` uses 4,835 and `LQNS` uses 985 on the 77 fixtures it can solve. The simple solver runs in 9x fewer iterations than `SolverLN` (reduced to 8.7x once the +2 offset is removed) and 1.8x fewer than `LQNS` on the corpus.

`SolverLN`’s `iter_min` floor is binding on 63 of 78 fixtures.

Of the 78 baseline fixtures, 63 report `ln_iters` = 53 exactly (the `iter_min` = 50 floor plus the 2 hard-reset bias and one verification iteration). The fifteen exceptions come from Partition B’s large task count chains and fanouts as well as Partition C’s saturated fanouts and a deep chain. On these, `SolverLN`’s convergence is topology-driven, however on the remaining 63, the floor is strictly binding and the iteration-count comparison is simply a comparison of detection rules rather than convergence rates. This motivates the `iter_min` ablation study in Section 4.6.2, investigating the underlying convergence behaviour and allowing the comparison to be more insightful. The per-partition counts behind these totals are reported in Table 4.2.

Per-partition aggregates.

As mentioned above, on every subpartition of A and most subpartitions of C, `SolverLN`’s mean iteration count is exactly 53, hitting the `iter_min` floor. Partition B varies on the larger task

Partition	N	Sim mean	LN mean	LQNS mean	Sim ÷ LN	Sim ÷ LQNS
A1 (single-class chains)	5	4.4	53.0	22.0	0.08x	0.20x
A2 (multi-class shared callee)	12	3.0	53.0	9.7	0.06x	0.31x
A3 (multi-server PS)	9	4.0	53.0	7.0	0.08x	0.57x
A4 (mixed topology)	10	5.2	53.0	8.2	0.10x	0.63x
A5 (activity-graph precedence)	7	3.6	53.0	7.5*	0.07x	0.48x*
B (scale axis)	10	8.3	109.8	19.9	0.08x	0.42x
C1 (fan-out callers)	6	11.0	53.0	7.3	0.21x	1.50x
C2 (INF-task hosts)	3	3.0	53.0	2.0	0.06x	1.50x
C3 (multi-class × multi-entry)	3	3.0	53.0	9.3	0.06x	0.32x
C4 (boundary cases)	3	3.7	53.0	7.7	0.07x	0.48x
C5 (saturated fan-out)	6	28.0	74.7	24.5	0.37x	1.14x
C6 (AND-fork join boundary)	2	3.5	53.0	29.0	0.07x	0.12x
C7 (deep chains)	2	7.5	54.5	32.0	0.14x	0.23x
TOTAL	78	6.9	62.0	12.8	0.11x	0.54x

*LQNS unavailable for 1 of 7 A5 fixtures

Table 4.2: Mean outer-iteration counts per solver per partition.

count fixtures designed to test the scalability of the solvers, where genuine topology-driven convergence carries **SolverLN**’s count above the floor. On this partition, the iteration gap widens further to a 13x ratio. LQNS sits between the two. On raw iteration counts, the simple solver beats LQNS on ten of thirteen partitions, losing on just C1, C2 and C5. On these cases, C2 (2 vs 3) and C5 (just 1.14x) are trivial, but C1 is more informative. The diagnosis is that our fixed **callerFansOut**-gated $\alpha_Z = 0.5$ damping is on from sweep 1, slowing convergence by design as the price of keeping the saturated fan-out fixtures off the iteration limit (Table 3.1). LQNS avoids this with adaptive under-relaxation, only engages once oscillation is detected. This is therefore one case where omission of a more complex mechanism costs iterations rather than saving them, marking the limit of the chapter’s wider claim.

Per-fixture distribution.

Figure 4.2 visualises each fixture’s iteration count across the three solvers, further strengthening the previous findings’ claims. The **SolverLN** points cluster at 53 because of the **iter_min** floor on most partitions. The **SolverLNSimple** points are lowest for the majority of fixtures, while the LQNS counts typically sit between the two.

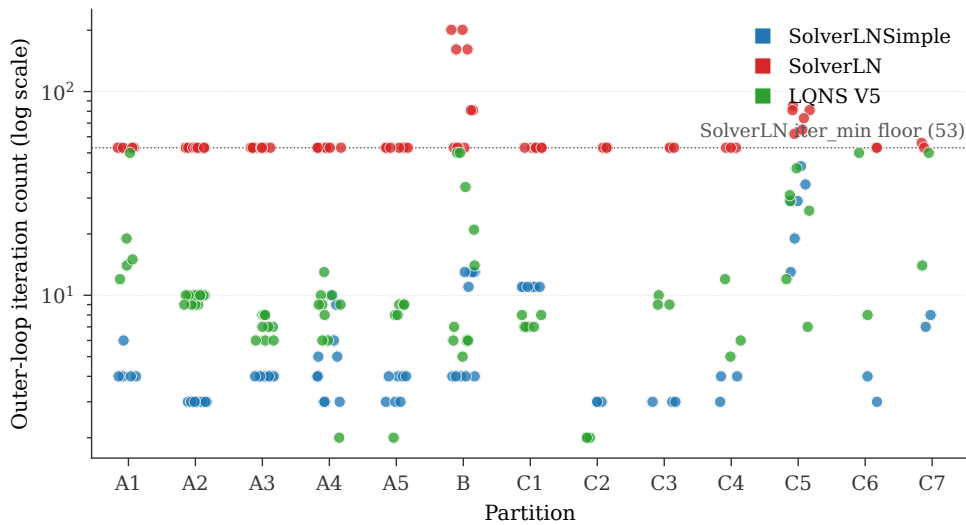


Figure 4.2: Per-fixture outer-loop iteration counts for the three solvers.

4.5 Runtime comparison

Iteration-count savings translate into wall-clock savings only if per-iteration costs are comparable. This section reports the per-fixture runtime comparison across all three solvers. As iteration counts vary across solvers, we also look into the per-iteration costs. However, again, this shouldn't be an exact comparison due to two asymmetries. Firstly, the LQNS time includes the REST round-trip to the Docker container that hosts LQNS, contributing a measured median 20 ms per call. In the opposite direction however, LQNS is a mature C++ implementation with decades of performance tuning, whereas `SolverLNSimple` is a Java/Kotlin prototype with limited bespoke runtime optimisation, just the primitive MVA kernels and graph caches.

Aggregate wall-clock across the three solvers.

Across the 78-fixture corpus, `SolverLN` takes 2,917,118 ms, LQNS takes 23,431 ms (on the 77 fixtures it can solve) and `SolverLNSimple` takes 22,184 ms. This positions `SolverLNSimple` 131x faster than `SolverLN` and 1.06x faster than LQNS as measured, with this reducing to 0.99x once the ~20 ms REST round-trip overhead is subtracted. The large gap to `SolverLN` sets the size of the runtime discrepancy our per-machinery decomposition later in the chapter characterises. Table 4.3 breaks these totals down by partition and per-fixture mean.

Partition	N	Sim total	LQNS total	LN total	Sim mean	LQNS mean	LN mean
A1 (single-class chains)	5	116	183	15,350	23	37	3,070
A2 (multi-class shared callee)	12	147	335	13,321	12	28	1,110
A3 (multi-server PS)	9	65	243	4,342	7	27	482
A4 (mixed topology)	10	309	282	23,952	31	28	2,395
A5 (activity-graph precedence)	7	1,402	234*	85,303	200	39*	12,186
B (scale axis)	10	18,649	20,928	2,631,495	1,865	2,093	263,150
C1 (fan-out callers)	6	103	175	7,326	17	29	1,221
C2 (INF-task hosts)	3	27	91	3,903	9	30	1,301
C3 (multi-class × multi-entry)	3	39	83	3,215	13	28	1,072
C4 (boundary cases)	3	19	79	1,531	6	26	510
C5 (saturated fan-out)	6	638	663	78,978	106	110	13,163
C6 (AND-fork join boundary)	2	633	76	44,929	316	38	22,464
C7 (deep chains)	2	37	59	3,473	18	30	1,736
TOTAL	78	22,184	23,431	2,917,118	284	304	37,399

LQNS unavailable for 1 of 7 A5 fixtures.

Table 4.3: Per-partition wall-clock totals and per-fixture means, all values in milliseconds.

Per-partition wall-clock concentrates on Partition B.

Partition B alone is 90.2% of `SolverLN`'s, 89.3% of LQNS's and 84.1% of `SolverLNSimple`'s corpus total, identifying the partition's per-layer MVA solve as its cost bottleneck. The ~20 ms LQNS floor visible in the eleven non-B/C6 partitions is the REST round-trip overhead mentioned. Focusing on partition B gives us a cleaner comparison of the solvers' per-iteration cost as REST calls, JIT state, and other sources of noise impact is minimised. When comparing partition B runtimes, we see `SolverLNSimple` as 1.12x faster than LQNS. Figure 4.3 plots the per-fixture runtimes across all three solvers.

Per-iteration runtime on Partition B across all three solvers.

Because iteration counts vary across solvers, the load-bearing comparison is per-iteration rather than aggregate (Table 4.4). LQNS wins on the five chain fixtures and `SolverLNSimple` wins on the five fanouts.

The `SolverLN`/`SolverLNSimple` ratio shows `SolverLNSimple` 7.2-29.3x cheaper per iteration than `SolverLN` on every Partition B fixture. Combined with the previous section's iteration-

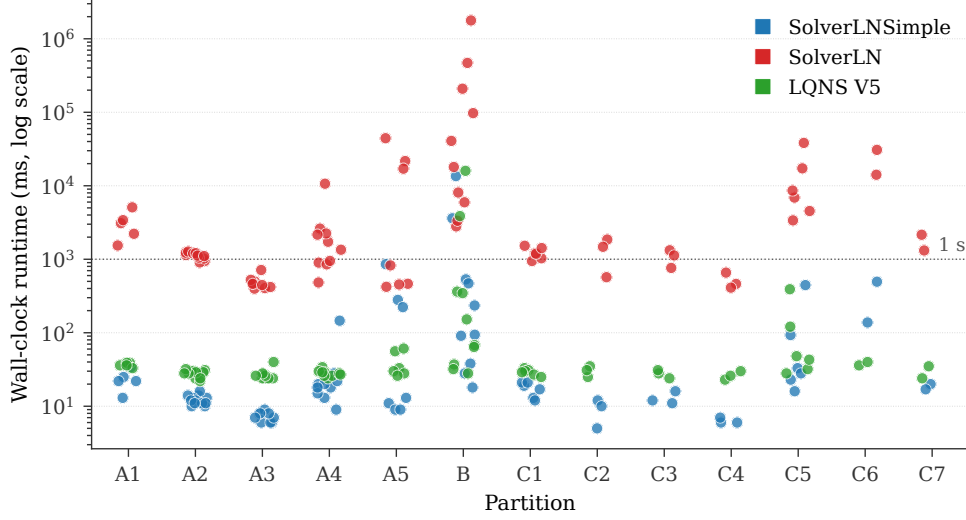


Figure 4.3: Per-fixture wall-clock runtime for the three solvers.

Fixture	LN it	LN ms/it §	LQNS it	LQNS ms/it	Sim it	Sim ms/it	LN/Sim	LQNS/Sim
B_scaleAxis_tasks5_chain	53	52.81	14	2.00	4	4.50	11.7x	0.44x
B_scaleAxis_tasks5_fanout	53	63.02	6	5.33	13	2.15	29.3x	2.48x
B_scaleAxis_tasks10_chain	53	112.5	21	1.76	4	9.50	11.8x	0.19x
B_scaleAxis_tasks10_fanout	53	152.6	7	9.71	13	7.00	21.8x	1.39x
B_scaleAxis_tasks20_chain	81	222.2	34	1.88	4	23.50	9.5x	0.08x
B_scaleAxis_tasks20_fanout	81	503.6	6	60.50	13	41.00	12.3x	1.48x
B_scaleAxis_tasks40_chain	161	604.9	50	3.04	4	58.75	10.3x	0.05x
B_scaleAxis_tasks40_fanout	161	2,916.7	6	645.0	13	277.6	10.5x	2.32x
B_scaleAxis_tasks60_chain	201	1,041.5	50	6.92	4	117.5	8.9x	0.06x
B_scaleAxis_tasks60_fanout	201	8,836.8	5	3,193.6	11	1,230.3	7.2x	2.60x

Table 4.4: Per-iteration wall-clock on Partition B.

count savings, this is the first evidence that suggests **SolverLNSimple**’s lead over **SolverLN** extends beyond convergence detection and into raw per-iteration cost, and the gap reflects genuine algorithmic machinery.

The LQNS/**SolverLNSimple** ratio inverts across the two shapes, 0.05x–0.44x for chains and 1.39x–2.60x for fan-outs, reflecting a difference in decomposition. **SolverLNSimple** splits the model one layer per task and processor, so every callee is a single-class submodel, width never enters any MVA’s class count, paying for depth in per-sweep work and for width in iteration count. LQNS keeps narrow-deep chain submodels cheap by paying for depth in iterations instead, but its per-iteration solve grows steeply with fan-out width.

SolverLNSimple’s activity graph runtime gap is not a per-iteration solve cost.

Partition A5 loses to LQNS on wall-clock greatly (200 ms mean vs 39 ms). However, of A5’s 1,402 ms **SolverLNSimple** total, 1,260 ms of it is accounted for by the three AND-fork fixtures, despite all converging in three iterations, and the other A5 fixtures converging in 2–3 ms. The partition-level “activity graphs are slow” reading is therefore false, and the cost is specific to the AND-fork→AND-join construct. Table 4.5 separates construction from iteration cost for representative A5 fixtures.

A profiling run shows the coupled-MVA fixed point is flat at 3.5–5.6 ms for *every* A5 fixture, and the entire delay is in construction, rising from ~18 ms on the OR-fork fixtures to 225–894 ms on the AND-fork fixtures. We can therefore determine that the iterative solver itself, including the $E[\max]$ join-delay machinery of the AND-fork throughput correction, is cheap, and it is instead the ensemble construction, that we have inherited from **SolverLN** that is expensive.

We see that construction calls **getStruct** on each per-task submodel, whose cost is dominated

A5 fixture	getEnsemble ms	iterate ms	LQNS ms
combinedDag	894	4.4	—
andForkMultiAct	276	3.7	33
andForkThreeWay	225	3.5	30
orForkThreeWay	18	5.0	56
orForkUneven	19	5.6	61

Table 4.5: Construction versus iteration wall-clock for representative A5 fixtures (separate instrumented profiling, warm JVM, $N = 100$). The fixed-point solve is flat; the AND-fork penalty is entirely in `getEnsemble`.

by the visit-ratio linear solves (`dtmc_solve`, `Matrix.solveSafe`). Fork/join branches give a richer routing matrix than other paradigms, so this generic refresh scales. LQNS, a native LQN solver, never builds a flat queueing network per task, so construction time stays low. We therefore get penalised for using `SolverLN`’s generic ensemble construction machinery.

LQNS net of REST round-trip overhead.

We instrument `SolverLQNS` to record per-call `rest_call_ms` (the HTTP block enclosing the remote solve) and `lqns_elapsed_ms` (the kernel solve, parsed from the LQXO). Their difference gives a median REST round-trip overhead of 20 ms per call. Subtracting this from each of the 77 solvable LQNS results, the LQNS corpus total drops from 23,431 ms to 21,891 ms, tightening `SolverLNSimple` $\div$ LQNS from 1.06x to 0.99x. We see that whilst most partitions lose their floor and drop below `SolverLNSimple`, partition B and C5 stay relatively unchanged as they are dominated by kernel time. Figure 4.4 re-plots the per-fixture runtimes with this overhead removed from the LQNS results.

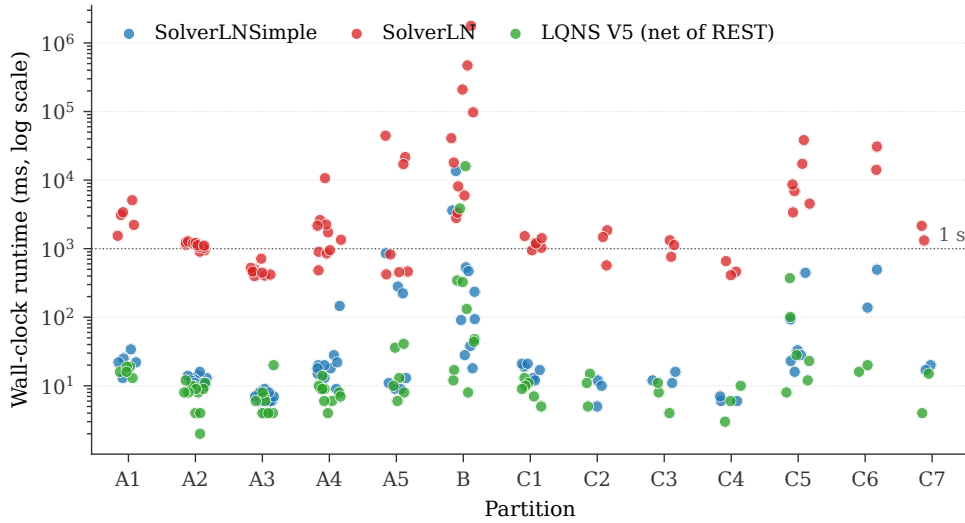


Figure 4.4: Figure 4.3 with LQNS V5 (green) net of REST overhead. Sim and LN unchanged.

4.6 SolverLN ablations

We hypothesise that the runtime and iteration-count gaps of the previous two sections are produced by specific `SolverLN` machinery that `SolverLNSimple` omits. We isolate the two largest contributors by disabling each inside `SolverLN` and re-measuring against the baseline. *Interlock correction* is a pure runtime-surface cost on this corpus, not affecting iteration count or output, but adding per-iteration work that grows with fan-out breadth. *Convergence control* dominates the iteration-count gap, but carries a small accuracy benefit.

4.6.1 Interlock correction

SolverLN

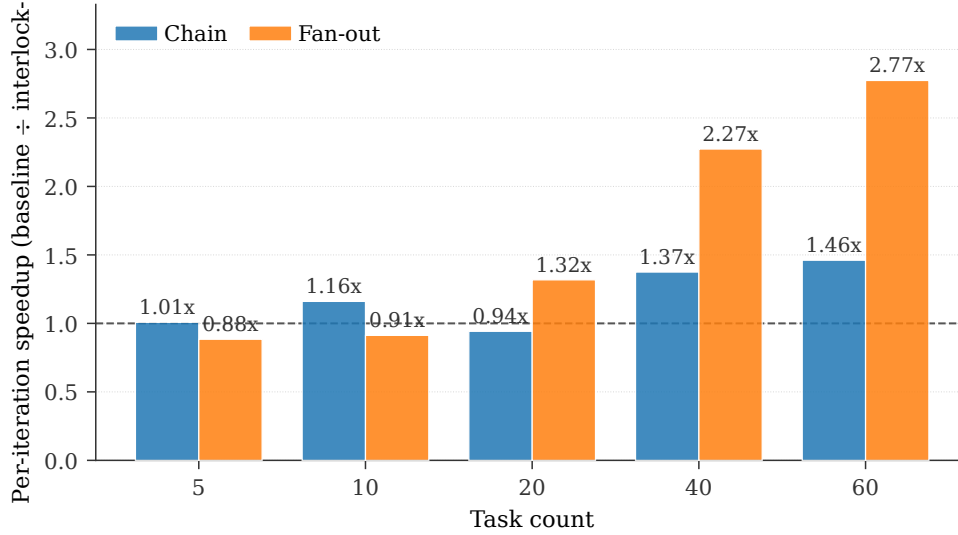


Figure 4.5: Per-iteration speedup from disabling the interlock subsystem on Partition B, by task count and topology.

Fixture	intlk-on	intlk-off	Δ (contrib)	on iters	off iters
directInterlock_n8	3.23%	3.96%	+0.73 pp	53	53
directInterlock_n16	0.86%	1.05%	+0.19 pp	53	53
transitiveInterlock_n8	2.98%	3.26%	+0.28 pp	83	83
multiclassInterlock	4.86%	6.27%	+1.41 pp	53	68

Table 4.7: Worst-case relative difference vs the `lqsim` ground truth on small diamond-shaped fixtures that trigger the interlock correction.

Two limitations we came across were a bug in `SolverLN`, where both `interlock-on` and `interlock-off` showed a $\sim 50\%$ error for some fixtures. We are also only able to test accuracy on small fixtures as otherwise it would become too costly to run the `lqsim` ground truth. However, this means we do not know if the correction’s contribution grows with scale.

4.6.2 Convergence control

`SolverLN.converged(it)` decides when the fixed-point iteration may stop. It bundles four pieces of machinery: an `iter_min` floor ($= \max(2 \cdot E, \text{ceil}(\text{iter_max}/4))$) that delays convergence until a minimum iteration count, a sliding-window weighted average over `Q/U/R/T` that smooths per-iteration metric noise, a three-iteration confirmation and a hard-reset verification pass. `SolverLNSimple` replaces the entirety with a single-iteration test, stopping the moment the per-layer throughput delta falls below `iter_tol`. This is the single largest contributor to the iteration-count gap. Disabling it, again, carries a small accuracy cost.

Disabling complex convergence logic cuts iteration count by 65%.

The corpus total drops from 4,835 to 1,672 outer iterations ($2.89\times$). The reduction concentrates on the floor, as 65 of the 78 fixtures sit at exactly 53 iterations but now collapse to single digits, exposing their true convergence point. The thirteen exceptions are Partition B’s large chains and fan-outs and Partition C’s saturated fan-outs and deep chains, where convergence is genuinely topology-driven above the floor, where the gap closes by $\sim 50\%$ rather than $\sim 80\%$. However even at 1,672, `SolverLN` still sits $3.1\times$ above `SolverLNSimple`’s 539, so the convergence machinery accounts for most but not all of the gap. Table 4.8 gives the per-partition breakdown and the resulting gap closure.

Partition	N	Sim	Baseline LN	Conv-off LN	Reduction	Gap closure
A1	5	22	265	74	3.58x	78.6%
A2	12	36	636	194	3.28x	73.7%
A3	9	36	477	108	4.42x	83.7%
A4	10	52	530	135	3.93x	82.6%
A5	7	25	371	70	5.30x	87.0%
B	10	83	1,098	521	2.11x	56.8%
C1	6	66	318	96	3.31x	88.1%
C2	3	9	159	30	5.30x	86.0%
C3	3	9	159	40	3.98x	79.3%
C4	3	11	159	32	4.97x	85.8%
C5	6	168	448	309	1.45x	49.6%
C6	2	7	106	16	6.62x	90.9%
C7	2	15	109	47	2.32x	66.0%
TOTAL	78	539	4,835	1,672	2.89x	73.6%

Table 4.8: Per-partition outer-iteration counts with the convergence control disabled, and the resulting closure.

A small accuracy cost.

The smoothing arms are what `SolverLNSimple` gives up for speed. Measuring `SolverLN`’s own per-cell output with and without the convergence machinery, we see that 7 of the 78 fixtures drift by more than 5% on some cell, ranging up to 32% (Table 4.9). The drift concentrates on cells that oscillate around the fixed point (per-class shared-callee queue lengths, low-demand response times, asymmetric multi-entry residence times). These are cells where the moving average smoothing would usually have an effect. `SolverLN`’s convergence detection takes the windowed-maximum error rather than the current-iteration error. Without it, detection can fire on a one-off low during an oscillation phase, with the response characterising whatever phase the iteration was in. The accuracy cost is from the moving-average smoothing and hard-reset arm, as the `iter_min` floor just delays convergence to the same fixed point. The correctness results (Section 4.3) show `SolverLNSimple` still lands within tolerance of a reference on these fixtures, so the omission is acceptable on this corpus, but we do recognise the small accuracy cost. It is also unfair to take these drifts at face value, as `SolverLN` was not built with a run like this in mind.

Fixture	Part.	Drift	Worst cell (base $\rightarrow$ conv-off)	Mechanism
A2_multiclassA_2refs_loadHigh_N50	A2	32.0%	R2.qlen (49.3 $\rightarrow$ 65.1)	per-class shared-callee oscillation
C4_lowDemand	C4	28.2%	E1.respt (0.052 $\rightarrow$ 0.067)	low-load tolerance-boundary oscillation
C3_multiclassMultiEntry_asymmetric	C3	15.8%	AS1.residt (1.58 $\rightarrow$ 1.83)	asymmetric multi-entry shared callee
A2_multiclassA_2refs_loadHigh_N20	A2	12.6%	R2.qlen (19.3 $\rightarrow$ 21.8)	per-class shared-callee oscillation
A2_multiclassA_2refs_loadLow_N50	A2	12.3%	R1.qlen (48.0 $\rightarrow$ 53.9)	per-class shared-callee oscillation
C5_satFanoutCaller_w2	C5	7.4%	TS1.qlen (3.78 $\rightarrow$ 4.06)	saturated fan-out caller oscillation
C7_deepChainMultiCall	C7	6.4%	T2.residt (0.30 $\rightarrow$ 0.32)	deep multi-call chain oscillation

Table 4.9: The seven fixtures whose `SolverLN` output drifts by more than 5% when the convergence control is disabled.

4.6.3 The residual algorithmic difference

The convergence machinery does not explain the whole gap in runtime and iteration count as with it disabled, `SolverLN` still uses 1,672 iterations against `SolverLNSimple`’s 539. The cut therefore closes $(4,835 - 1,672)/(4,835 - 539) = 73.6\%$ of the iteration-count gap, where the remaining $\approx 26\%$ (1,133 iterations corpus-wide) is genuine algorithmic difference between the two solvers, not just convergence machinery. Per partition, the closure ranges from $\approx 50\%$ on Partition B and C5, where `SolverLN`’s convergence is genuinely slower than the simple solver’s,

to $\approx 90\%$ on C6 (Figure 4.6). The residual comes from the simple solver’s three structural choices that the contribution chapter names: predicate-gated α_Z damping fired only where it is needed rather than `SolverLN`’s unconditional blend, structural service-time propagation rather than `*_prev` snap-back arrays to handle failures, and single-iteration convergence detection providing looser guarantees rather than three-iteration confirmation.

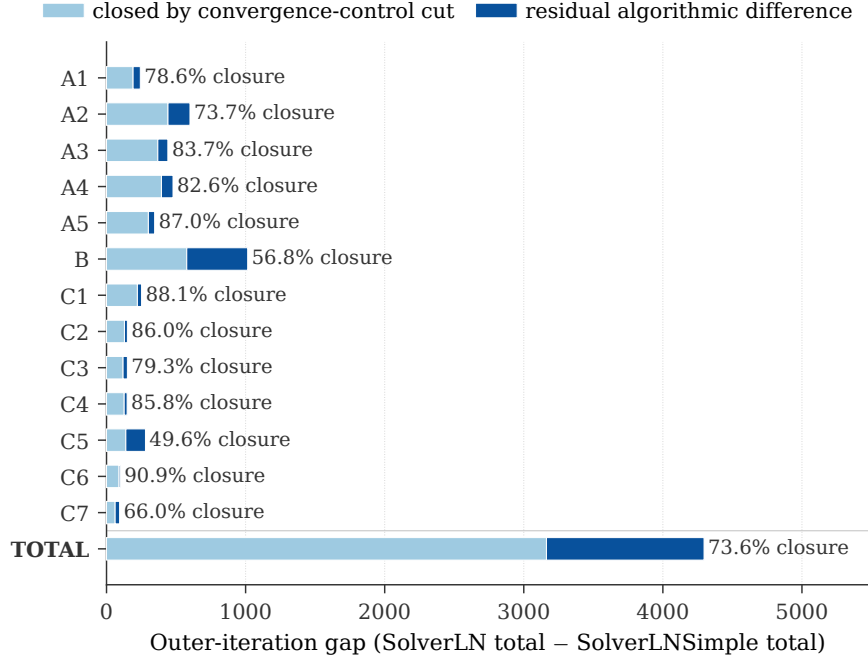


Figure 4.6: Decomposition of the `SolverLN`-vs-`SolverLNSimple` iteration-count gap per partition. Each bar’s total length is the per-partition gap (`baseline LN – Sim`), split into a light portion closed by disabling the convergence control and a dark residual algorithmic difference. The aggregate split is 73.6% / 26.4%; per-partition closure ranges from $\approx 50\%$ (B, C5) to $\approx 90\%$ (C6).

4.7 The accuracy boundary

This section maps where the `SolverLNSimple` and `SolverLN` diverge. Of the 78 fixtures the 57 `match-ln` are graded in Section 4.3 and the other 21 are characterised here. Fourteen are divergent-by-design, where they match LQNS instead when the reference solvers diverge. Seven match neither reference, whose structural causes were previously named in correctness Section 4.3. Therefore every divergence is accounted for and none disagree with our correctness claims.

A host-layer modelling divergence that matches LQNS. The fourteen `match-lqns-only` fixtures (six C1 fan-out callers, five B fan-out fixtures, three C5 saturated fan-outs) sit 28%–81% from `SolverLN` but $\leq 7.7e-02$ from LQNS (Table 4.10). The cause is a difference in host-layer modelling where at moderate-to-high per-server utilisation `SolverLN` collapses the host residence to its cheap M/M/ ∞ -like $R \approx D$ approximation, dropping the queueing term, while `SolverLNSimple` keeps the closed-PFQN M/M/c queueing tail. The gap widens with fan-out width and grows monotonically with task count, while C1 additionally fires the `LqnGraph.callerFansOut`-gated fixed- α damping. Whether keeping the queueing tail is more accurate or spurious in trees sized for M/M/ ∞ behaviour can only be settled by discrete-event simulation (`lqsim`), which the chapter does not do. The narrower, sound claim is that `SolverLNSimple` reproduces the established tool’s host-layer choice where `SolverLN` departs from it. This is an interesting case however, as it is `SolverLNSimple` that opts for more machinery and rigorous

computation.

Fixture	Partition	sim vs LN max rel diff	Mechanism
C1_fanOut2_caller_c1	C1	0.294	Fan-out caller; predicate-gated (α) damping disagreement, widening with fan-out width
C1_fanOut2_caller_c2	C1	0.294	
C1_fanOut3_caller_c1	C1	0.362	
C1_fanOut3_caller_c2	C1	0.362	
C1_fanOut4_caller_c1	C1	0.303	
C1_fanOut4_caller_c2	C1	0.303	
B_scaleAxis_tasks5_fanout	B	0.281	M/M/ ∞ -vs-M/M/c host approximation at moderate ρ ; grows monotonically with task count
B_scaleAxis_tasks10_fanout	B	0.466	
B_scaleAxis_tasks20_fanout	B	0.594	
B_scaleAxis_tasks40_fanout	B	0.723	
B_scaleAxis_tasks60_fanout	B	0.808	
C5_satFanout_tasks5	C5	0.441	Same M/M/ ∞

The only partitions where it loses to LQNS on are C1, C2, and C5, which are either trivial or as a result of fixed-vs-adaptive damping (Section 4.4).

H3 (runtime): confirmed. 22,184 ms against SolverLN’s 2,917,118 ms (131x) and LQNS’s 23,431 ms (1.06x faster as measured but 0.99x once the REST round-trip is removed, still a good win for a Java-vs-C++ comparison) (Section 4.5).

H4 (interlock is a runtime-surface cost): confirmed. Disabling the correction leaves every fixture’s iteration count and output metric unchanged while removing per-iteration cost that grows up to 2.77x on the widest fanout case (Table 4.6). On a corpus that never triggers the correction, it is a pure per-iteration cost. On models that do trigger it, we see a small accuracy difference (Section 4.6.1).

H5 (convergence control dominates the iteration-count gap): confirmed. Disabling it cuts the corpus from 4,835 to 1,672 iterations (2.89x), closing 73.6% of the gap. The reduction concentrates on the floor, where most of the fixtures pinned at 53 iterations collapse to single digits, exposing their true convergence point (Section 4.6.2).

H6 (convergence control accuracy cost): confirmed and quantified. Seven of 78 fixtures drift by more than 5% when the smoothing is removed, up to 32%, all on cells that oscillate around the fixed point (Table 4.9). The cost is real, but mostly concentrated and small (Section 4.6.2).

H7 (residual remains): confirmed at 26.4%. With convergence control off, SolverLN still uses 1,672 iterations against the SolverLNSimple’s 539, a residual the convergence machinery does not explain, which Section 4.6.3 attributes structurally to the simple solver’s coupling and damping choices.

4.8.2 What the chapter does not claim

The chapter does not claim SolverLN’s machinery is redundant in general. The interlock correction is unexercised here only because no corpus fixture instantiates a shared server with two or more distinct client tasks. The diamond probe (Table 4.7) shows it does real accuracy work where it fires. The convergence smoothing carries a measurable accuracy cost when removed (Table 4.9), so it too is doing useful work on oscillation-prone fixtures. The claim is the narrower one. On this corpus, omitting these machineries recovers most of the iteration-count difference and a large share of the per-iteration runtime at a small, bounded, quantified accuracy cost.

We also don’t try to claim correctness in the saturated and AND-fork regimes the seven **match-neither** fixtures occupy. The three C5 saturated fan-outs ($\rho \approx 0.99$) and two C6 AND-fork fixtures push the simple solver’s closed-form kernels past the regimes they are derived for, and the two C7 deep all-Exp chains converge to a steady state differing from both references (Section 4.7). These are deliberately adversarial cases outside the claimed scope, surfaced rather than hidden. The boundary section names the concrete extension each failure would need.

Finally, the chapter does not take reference solver outputs at face value. It uses them as a guideline for structural comparison and correctness rather than as ground truth, so the host-layer disagreement it surfaces is characterised, not adjudicated.

Three of these gaps are also the natural next steps. The 26.4% residual is attributed structurally rather than measured, so ablating the three machineries we claim to be residual would convert that attribution into a quantified figure. The host-layer disagreement is characterised rather than resolved, where an `lqsim` ground-truth study at corpus scale would be required to adjudicate it. And on the runtime side, the activity-graph construction cost (Section 4.5, Table 4.5) is inherited SolverLN ensemble-construction machinery rather than solver cost, making it a clear target for engineering.

4.8.3 Closing

We see that our SolverLNSimple wins on both iteration count and runtime, showing the benefits of a simplified, less general solver. The chapter has characterised the spine claim’s three-quarters / one-quarter split on the corpus the build surfaced, where convergence control accounts for

most of the iteration-count gap and the interlock correction for a fan-out-scaling share of the per-iteration runtime, while a quarter of the iteration-count gap remains a genuine algorithmic difference. The Discussion chapter that follows takes this further, considering the residual and asking whether a solver that omits the residual convergence-smoothing machinery and the interlock random walk is a defensible architecture for closed LQN solvers in general, or a localised result valid only on the corpus this chapter measured.

Chapter 5

Conclusions

The Evaluation chapter closes by asking what its result means beyond the corpus it measured and whether a solver that omits **SolverLN**’s convergence smoothing and interlock random walk is a defensible architecture for closed LQN models in general, or a localised finding valid only on the fixtures we built. This chapter answers that question, assessing the evidence against the project’s two contributions and separating the simple solver’s deliberate scope decisions from the limits of the bottom-up methodology. Every quantitative claim is drawn from the Evaluation chapter (Chapter 4) or the build (Chapter 3) such that this chapter is an assessment rather than a measurement.

5.1 Assessing the result

The spine claim the project commits to is that the difference between **SolverLN** and **SolverLNSimple** on closed LQN models is real, but decomposable across both iteration count (approximately three-quarters is **SolverLN**’s convergence control rather than a slower fixed point, while the remaining quarter genuine algorithmic difference) and on runtime (the interlock correction imposes per-iteration cost that grows with fan-out breadth while touching neither iteration count nor accuracy on this corpus). The price for omitting both machineries is a small, bounded accuracy cost on a minority of fixtures, each of which still lands within a reference’s tolerance. The evaluation confirms this claim, and the assessment below grades the project’s two contributions against it.

The construction is correct where it claims to be. Of the 78 fixtures, 71 pass: every one of Partition A’s 43 positive-evidence fixtures matches **SolverLN** to within the 5% bar on every metric, and every Partition B fixture matches one of the two references (Section 4.3, Table 4.1). The seven **match-neither** failures all sit in the three adversarial Partition C subpartitions (three saturated fan-outs, two AND-fork joins, two deep all-Exp chains) that were constructed to push the closed-form kernels past the regimes they are derived for. The largest residual from the positive-evidence set was a multi-class $\times$ multi-entry A4 fixture, falling just within the 5% tolerance, evidencing this class as the corpus’s most accuracy-sensitive sub-partition (Section 4.7). Therefore, correctness holds in its intended scope.

The performance measure decomposition is delivered. On iteration count, **SolverLNSimple** uses 539 outer iterations against **SolverLN**’s 4,835 (9 $\times$) and LQNS’s 985 (1.8 $\times$). Disabling **SolverLN**’s convergence control cuts its corpus total to 1,672, closing 73.6% of the gap by allowing it to converge to its true fixed point and leaving a 26.4% residual of 1,133 iterations attributed structurally (Section 4.6.3, Table 4.8). The floor is structural rather than incidental, dominating on 63 of 78 fixtures, so on those fixtures disabling the floor is essentially disabling the gap. On runtime, the 131 $\times$ wall-clock advantage over **SolverLN** (22,184 ms against 2,917,118 ms) is carried by the interlock correction, where the gap decreases to 59 $\times$ (1,310,156 ms) once it is disabled, while leaving every iteration count and output metric unchanged (Section 4.6.1, Table 4.6). Against LQNS, the runtime is comparable (1.06 $\times$ as measured, 0.99 $\times$ net of the

REST round-trip).

The accuracy price is quantified. `SolverLNSimple` is not a better algorithm than `SolverLN`, it is instead that the difference is specific, named machinery that does not have a large impact on this corpus, made visible because the bottom-up build deliberately omitted it. The contribution is the construction and characterisation, not a claim of algorithmic superiority. Disabling `SolverLN`'s convergence smoothing moves 7 of 78 of its own outputs off the fixed point by more than 5%, up to 32%, all on cells that oscillate around the fixed point (Section 4.6.2, Table 4.9). This is the cost `SolverLNSimple` accepts for omitting the moving-average machinery, allowing a real but concentrated accuracy cost on a tenth of the corpus, and small enough that the simple solver still lands within a 5% tolerance of the reference on every fixture in the corpus.

5.2 Is `SolverLNSimple` a defensible architecture?

For the class of closed LQN model the corpus exercises, the stripped architecture is defensible. The two machineries the simple solver omits do nothing the corpus requires, such that the `iter_min` floor only delays convergence, while the interlock correction would never actually fire, so removing them recovers most of the iteration-count gap and a large share of the per-iteration runtime at a bounded accuracy cost. The host-layer disagreement sharpens the point in the simple solver's favour, where on the fourteen `match-lqns-only` fan-out fixtures in the corpus, it is `SolverLNSimple` that keeps the rigorous closed-PFQN M/M/c queueing tail and tracks the mature LQNS, while `SolverLN` falls back to the cheaper M/M/ ∞ -like $R \approx D$ substitution (Section 4.7, Table 4.10), showing the simpler solver is not simpler by approximating more aggressively.

However, the result is still localised. The corpus never instantiates a server with two or more distinct client tasks, so it never fires the interlock correction. The probe showed us that where the correction does fire it is genuine accuracy machinery, improving the worst-case error against an `lqsim` ground truth by 0.19 to 1.41 percent (Section 4.6.1, Table 4.7). The convergence smoothing does real work on oscillation-prone fixtures, as its removal cost shows. The seven boundary failures also locate exactly where the closed-form kernels the solver uses break. The honest statement is therefore that the architecture is defensible *for the class it claims*, not in general. However, this corpus was built top-down from five measurement goals, deliberately separate from the fixtures that drove construction, so that evaluation is not shaped by construction, preventing the use of a survivorship artefact (Section 4.2.2).

5.3 The surfaced algorithmic candidates

The bottom-up methodology was expected to surface algorithmic choices worth investigation in their own right, the main one being a structural predicate-gated under-relaxation. The fan-out capability (Section 3.5) introduces a fixed- α damping (`Z_RELAX_ALPHA = 0.5`) gated on a static predicate, `LqnGraph.callerFansOut`, that is true when a caller has more than one distinct synchronous callee. The mechanism is distinct from `SolverLN`'s adaptive under-relaxation, which detects oscillation by sign-change, and pays three to four detection iterations before damping starts. On fan-out workloads, the false-saturation fixed-point locks in before adaptive damping arrives, whereas structural gating fires from the first sweep on the topology that needs it and is off elsewhere. Despite our prediction however, its trade-off is visible. On Partition C1, the fixed predicate-gated damping being always-on actually costs more iterations than LQNS's adaptive scheme as the damping prevents reaching the fixed point quickly. This is therefore the one case in this build where omitting the more complex mechanism costs rather than saves (Section 4.4, Table 3.1). A true evaluation towards the trade-off of approaches is noted as potential future work in Section 5.6, but as the candidate is simply surfaced and not established, the project does not claim it as a contribution.

We also had a bounce-sweep iteration order and considered it a second candidate, as we thought propagating values in both directions more quickly and reducing the effect of the per-

iteration overhead would lead to faster convergence, however this was not the case. The optimisation work re-examined sweep order and migrated to the elevator pattern the references already use (Section 3.7, Table 3.2), chosen for a like-for-like iteration comparison in the Evaluation chapter. Whilst we saw a decrease in iteration count for the bounce-sweep, this was simply due to it performing double the work per iteration, as the runtime showed very little performance difference.

5.4 Out of scope vs Out of reach

The simple solver is missing modelling constructs that `SolverLN` and `LQNS` support, falling into two categories.

Out of scope are constructs the bottom-up build has a clear shape introducing, where the only thing missing is a motivating fixture that forces the addition. Open-class arrivals on entries, call forwarding, full per-iteration threading of phase-2 background work and the interlock correction all sit within this category. Each would be a coupling-channel or post-hoc extension that would not disturb the inner MVA kernel.

Out of reach are constructs the methodology cannot introduce naturally, as they require changing the inner kernel, the coupling channel, or the per-layer solver dispatch. Cache-modelling would require new node types and per-iteration routing re-derivation [27], richer 3-moment APH (Acyclic Phase-Type) propagation needs a kernel that carries an SCV (Squared Coefficient of Variance) parameter rather than the mean-only closed-form kernel the simple solver uses [28], class-switching at iteration time interacts with the coupling structure in a way the per-layer client/host channel does not express [8], and the AND-fork case where sibling branches contend on a finite-server PS host needs Franks’s CCD overlap compensation [5] rather than the closed-form $E[\max]$ join.

The 26.4% residual we identify is attributed structurally to three choices the simple solver makes (predicate-gated damping, structural service-time propagation rather than snap-back arrays for failure recovery, and single-iteration convergence detection rather than three-iteration confirmation). However, we do not separately perform ablations on these as the project’s ablations were spent on the comparator side rather than self-ablation.

5.5 Field-level reflection

The project raises a small set of observations about LQN solver research as an area. Each is offered as a suggestion supported by the project’s findings, not as a general principle it has established.

The outer loop has been under-audited. The iteration-economy literature has decomposed the inner loop but not the outer one. Franks and Li’s One-Step MVA [23] collapses the inner AMVA convergence on the argument that the outer fixed point will invalidate any over-precise submodel solve [23]. The LN paper [15] describes its outer convergence test and elevator order but does not analyse the individual outer-loop machineries against the correctness-efficiency trade-off removing them exposes. Work that recognises specific outer-loop costs, such as surrogate-delay iteration and Flow-Equivalent Service Centre (FESC) evaluation [29], doesn’t perform per-machinery decomposition (Section 2.4.7). The project’s evaluation, isolating the convergence floor and the interlock walk, and pricing each, is a contribution toward that gap.

Accreted machinery propagates by porting. `SolverLN` inherits significant machinery from `LQNS` by direct porting (Section 2.4.6). If accuracy is the primary concern, porting of trusted machinery is the right thing to do, however it means that machinery once added for a specific hard case remains across generations of tools. The bottom-up build is a deliberate response to this pattern, instead starting from a base case and adding only what a specific model forces, rather than inheriting machinery and assuming it is necessary.

The methodological pattern. The shape of the work (a build from a base case, machinery added only when a model forces it, with the test corpus treated as an instrument that records

what each step had to handle) is what made the per-machinery attribution possible. The build's omissions are what let the difference against an established tool be pinned to named machinery rather than vague design choices. Whether the pattern transfers beyond LQN solvers is a question the project cannot answer. General-purpose ODE solvers, with decades of accreted heuristics around stiffness detection and error estimation [30], are a plausible candidate by analogy, however this is left open for future work.

5.6 Future work

The bottom-up methodology extends in three directions, each a continuation of the same build:

Feature parity with the established tools. The out-of-scope constructs of Section 5.4 come in when motivating fixtures force them. We suggest adding open-class arrivals on entries for external-traffic-driven systems, call forwarding for reverse-proxy topologies, full per-iteration

Bibliography

- [1] J. Lewis and M. Fowler, “Microservices: a definition of this new architectural term.” <https://martinfowler.com/articles/microservices.html>, 2014. Accessed: 12-06-26.
- [2] R. Heinrich, A. van Hoorn, H. Knoche, F. Li, L. E. Lwakatare, C. Pahl, S. Schulte, and J. Wettinger, “Performance engineering for microservices: Research challenges and directions,” in *Proceedings of the 2017 ACM/SPEC International Conference on Performance Engineering Companion*, pp. 223–226, 2017.
- [3] B. Urgaonkar, G. Pacifici, P. Shenoy, M. Spreitzer, and A. Tantawi, “An analytical model for multi-tier internet services and its applications,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 33, no. 1, pp. 291–302, 2005.
- [4] E. D. Lazowska, J. Zahorjan, G. S. Graham, and K. C. Sevcik, *Quantitative System Performance: Computer System Analysis Using Queueing Network Models*. Englewood Cliffs, NJ: Prentice Hall, 1984.
- [5] R. G. Franks, *Performance Analysis of Distributed Server Systems*. PhD thesis, Department of Systems and Computer Engineering, Carleton University, Ottawa, Canada, December 1999.
- [6] G. Franks, T. Al-Omari, M. Woodside, O. Das, and S. Derisavi, “Enhanced modeling and solution of layered queueing networks,” *IEEE Transactions on Software Engineering*, vol. 35, pp. 148–161, March/April 2009.
- [7] C. M. Woodside, “Tutorial introduction to layered modeling of software performance, edition 4.0,” tech. rep., Real-time and Distributed Systems Lab (RADS), Carleton University, May 2002. Available at <http://sce.carleton.ca/rads>.
- [8] F. Baskett, K. M. Chandy, R. R. Muntz, and F. G. Palacios, “Open, closed, and mixed networks of queues with different classes of customers,” *Journal of the ACM*, vol. 22, no. 2, pp. 248–260, 1975.
- [9] J. R. Spirn, “Queueing networks with random selection for service,” *IEEE Transactions on Software Engineering*, vol. SE-5, no. 3, pp. 287–289, 1979.
- [10] M. Reiser and S. S. Lavenberg, “Mean value analysis of closed multichain queueing networks,” *Journal of the ACM*, vol. 27, no. 2, pp. 313–322, 1980.
- [11] Y. Bard, “Some extensions to multiclass queueing network analysis,” in *Proceedings of the Third International Symposium on Modelling and Performance Evaluation of Computer Systems: Performance of Computer Systems* (M. Arató, A. Butrimenko, and E. Gelenbe, eds.), pp. 51–62, North-Holland, 1979. ISBN 0-444-85332-4.
- [12] C. M. Woodside, “Throughput calculation for basic stochastic rendezvous networks,” *Performance Evaluation*, vol. 9, no. 2, pp. 143–160, 1989.

- [13] C. M. Woodside, J. E. Neilson, D. C. Petriu, and S. Majumdar, “The stochastic rendezvous network model for performance of synchronous client-server-like distributed software,” *IEEE Transactions on Computers*, vol. 44, no. 1, pp. 20–34, 1995.
- [14] J. A. Rolia and K. C. Sevcik, “The method of layers,” *IEEE Transactions on Software Engineering*, vol. 21, pp. 689–700, September 1995.
- [15] G. Casale, Y. Gao, Z. Niu, and L. Zhu, “Ln: A flexible algorithmic framework for layered queueing network analysis,” *ACM Transactions on Modeling and Computer Simulation*, vol. 34, pp. 1–26, July 2024.
- [16] M. Reiser, “Mean-value analysis and convolution method for queue-dependent servers in closed queueing networks,” *Performance Evaluation*, vol. 1, no. 1, pp. 7–18, 1981.
- [17] K. M. Chandy and D. Neuse, “Linearizer: A heuristic algorithm for queueing network models of computing systems,” *Communications of the ACM*, vol. 25, no. 2, pp. 126–134, 1982.
- [18] J. Zahorjan, D. L. Eager, and H. M. Sweillam, “Accuracy, speed, and convergence of approximate mean value analysis,” *Performance Evaluation*, vol. 8, no. 4, pp. 255–270, 1988.
- [19] A. Seidmann, P. J. Schweitzer, and S. Shalev-Oren, “Computerized closed queueing network models of flexible manufacturing systems: A comparative evaluation,” *Large Scale Systems*, vol. 12, pp. 91–107, 1987.
- [20] J. A. Rolia, *Predicting the Performance of Software Systems*. PhD thesis, University of Toronto, Toronto, Canada, January 1992. Also issued as Technical Report CSRI-260, Computer Systems Research Institute.
- [21] G. Franks, P. Maly, M. Woodside, D. C. Petriu, A. Hubbard, and M. Mroz, *Layered Queueing Network Solver and Simulator User Manual*. Carleton University, RADS Lab, 2022.
- [22] G. Casale, “Integrated performance evaluation of extended queueing network models with line,” in *WSC ’20: Proceedings of the Winter Simulation Conference*, pp. 2377–2388, 2021.
- [23] G. Franks and L. Li, “Efficiency improvements for solving layered queueing networks,” in *ICPE ’12: Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering*, (Boston, MA), pp. 279–282, ACM, 2012.
- [24] T. Israr, M. Woodside, and G. Franks, “Interaction tree algorithms to extract effective architecture and layered performance models from traces,” *Journal of Systems and Software*, vol. 80, no. 4, pp. 474–492, 2007.
- [25] P. Heidelberger and K. S. Trivedi, “Analytic queueing models for programs with internal concurrency,” *IEEE Transactions on Computers*, vol. C-32, pp. 73–82, January 1983.
- [26] Y. Saad, *Iterative Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2nd ed., 2003.
- [27] Y. Gao, G. Casale, and R. Singhal, “Performance modeling of distributed data processing in microservice applications,” *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 9, no. 4, pp. 1–30, 2024. Article 14.
- [28] A. Bobbio, A. Horváth, and M. Telek, “Matching three moments with minimal acyclic phase type distributions,” *Stochastic Models*, vol. 21, no. 2–3, pp. 303–326, 2005.

- [29] F. Islam, D. C. Petriu, and M. Woodside, “Focused layered performance modelling by aggregation,” *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 7, no. 2–4, pp. 1–23, 2022.
- [30] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE suite,” *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997.

Declarations

Use of Generative AI

I acknowledge the use of generative AI tools, such as Claude, in this project.

GenAI is increasingly becoming an efficient tool in software engineering to support the implementation of features. Whilst it was used in my project, all ideas were my own and features were considered and constructed carefully, as my intentions for the project and domain knowledge I gathered over the project were ideas that could only I could motivate. However, I found it useful to get AI to regularly refactor and document my code to help maintain what eventually became a large codebase.

While writing my report, GenAI has been a valuable tool in formatting LaTeX, however once again it was not the right situation to use heavily in writing, as the project timeline and intentions, framing and domain knowledge are only things I could contribute.

GitLab repository link

<https://gitlab.doc.ic.ac.uk/as4322/line-dev-alex>

Appendix

The evaluation test suite

```
// =====
// Partition A1: Single-class chains
// =====

@Test @Timeout(180) public void A1_chain_d3_N50() { assertResultsMatchSolverLN(
    EvaluationSuite::A1_chain_d3_N50); }
@Test @Timeout(180) public void A1_chain_d5_N20() { assertResultsMatchSolverLN(
    EvaluationSuite::A1_chain_d5_N20); }
@Test @Timeout(180) public void A1_chain_d5_N50() { assertResultsMatchSolverLN(
    EvaluationSuite::A1_chain_d5_N50); }
@Test @Timeout(240) public void A1_chain_d5_N100() { assertResultsMatchSolverLN(
    EvaluationSuite::A1_chain_d5_N100); }
@Test @Timeout(240) public void A1_chain_d8_N50() { assertResultsMatchSolverLN(
    EvaluationSuite::A1_chain_d8_N50); }

// =====
// Partition A2: Multi-class with shared callees
// =====

@Test @Timeout(180) public void A2_multiclassA_2refs_loadLow_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassA_2refs_loadLow_N20); }
@Test @Timeout(180) public void A2_multiclassA_2refs_loadHigh_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassA_2refs_loadHigh_N20); }
@Test @Timeout(180) public void A2_multiclassA_2refs_loadLow_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassA_2refs_loadLow_N50); }
@Test @Timeout(180) public void A2_multiclassA_2refs_loadHigh_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassA_2refs_loadHigh_N50); }
@Test @Timeout(180) public void A2_multiclassB_2refs_demands_3to1_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassB_2refs_demands_3to1_N20); }
@Test @Timeout(180) public void A2_multiclassB_2refs_demands_5to1_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassB_2refs_demands_5to1_N20); }
@Test @Timeout(180) public void A2_multiclassB_2refs_demands_3to1_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassB_2refs_demands_3to1_N50); }
@Test @Timeout(180) public void A2_multiclassB_2refs_demands_5to1_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassB_2refs_demands_5to1_N50); }
@Test @Timeout(180) public void A2_multiclassC_2refs_think_2to1_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassC_2refs_think_2to1_N20); }
@Test @Timeout(180) public void A2_multiclassC_2refs_think_4to1_N20() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassC_2refs_think_4to1_N20); }
@Test @Timeout(180) public void A2_multiclassC_2refs_think_2to1_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassC_2refs_think_2to1_N50); }
@Test @Timeout(180) public void A2_multiclassC_2refs_think_4to1_N50() {
    assertResultsMatchSolverLN(EvaluationSuite::A2_multiclassC_2refs_think_4to1_N50); }

// =====
// Partition A3: Multi-server processors
// =====
```

```

@Test @Timeout(180) public void A3_multiserver_c2_loadLow()      { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c2_loadLow);    }
@Test @Timeout(180) public void A3_multiserver_c2_loadMedium() { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c2_loadMedium); }
@Test @Timeout(180) public void A3_multiserver_c2_loadHigh()   { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c2_loadHigh);   }
@Test @Timeout(180) public void A3_multiserver_c4_loadLow()     { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c4_loadLow);     }
@Test @Timeout(180) public void A3_multiserver_c4_loadMedium() { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c4_loadMedium); }
@Test @Timeout(180) public void A3_multiserver_c4_loadHigh()   { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c4_loadHigh);   }
@Test @Timeout(180) public void A3_multiserver_c8_loadLow()     { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c8_loadLow);     }
@Test @Timeout(180) public void A3_multiserver_c8_loadMedium() { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c8_loadMedium); }
@Test @Timeout(180) public void A3_multiserver_c8_loadHigh()   { assertResultsMatchSolverLN(
    EvaluationSuite::A3_multiserver_c8_loadHigh);   }

// =====
// Partition A4: Mixed topology
// =====

@Test @Timeout(240) public void A4_threeTier_multiclass()      { assertResultsMatchSolverLN(
    EvaluationSuite::A4_threeTier_multiclass);    }
@Test @Timeout(240) public void A4_multiclass_multiserver()   { assertResultsMatchSolverLN(
    EvaluationSuite::A4_multiclass_multiserver);   }
@Test @Timeout(240) public void A4_multiclass_multientry()    { assertResultsMatchSolverLN(
    EvaluationSuite::A4_multiclass_multientry);    }
@Test @Timeout(240) public void A4_chainPlusFanout()           { assertResultsMatchSolverLN(
    EvaluationSuite::A4_chainPlusFanout);           }
@Test @Timeout(240) public void A4_webApplication()            { assertResultsMatchSolverLN(
    EvaluationSuite::A4_webApplication);            }
@Test @Timeout(240) public void A4_databaseBackend()           { assertResultsMatchSolverLN(
    EvaluationSuite::A4_databaseBackend);           }
@Test @Timeout(240) public void A4_threeTier_sequence()       { assertResultsMatchSolverLN(
    EvaluationSuite::A4_threeTier_sequence);       }
@Test @Timeout(240) public void A4_parallelService_andFork()  { assertResultsMatchSolverLN(
    EvaluationSuite::A4_parallelService_andFork);  }
@Test @Timeout(240) public void A4_retryLoop()                 { assertResultsMatchSolverLN(
    EvaluationSuite::A4_retryLoop);                 }
@Test @Timeout(240) public void A4_replyThenLog()              { assertResultsMatchSolverLN(
    EvaluationSuite::A4_replyThenLog);              }

// =====
// Partition A5: Activity-graph capability coverage
// =====

@Test @Timeout(180) public void A5_orForkUneven()               { assertResultsMatchSolverLN(
    EvaluationSuite::A5_orForkUneven);               }
@Test @Timeout(180) public void A5_orForkThreeWay()            { assertResultsMatchSolverLN(
    EvaluationSuite::A5_orForkThreeWay);            }
@Test @Timeout(180) public void A5_andForkThreeWay()           { assertResultsMatchSolverLN(
    EvaluationSuite::A5_andForkThreeWay);           }
@Test @Timeout(180) public void A5_andForkMultiAct()           { assertResultsMatchSolverLN(
    EvaluationSuite::A5_andForkMultiAct);           }
@Test @Timeout(180) public void A5_seqMultiEntry()            { assertResultsMatchSolverLN(
    EvaluationSuite::A5_seqMultiEntry);            }
@Test @Timeout(180) public void A5_replyAtBound()              { assertResultsMatchSolverLN(
    EvaluationSuite::A5_replyAtBound);              }
@Test @Timeout(240) public void A5_combinedDag()               { assertResultsMatchSolverLN(
    EvaluationSuite::A5_combinedDag);               }

```

```

// =====
// Partition B: Scale axis for F24
// =====

@Test @Timeout(180) public void B_scaleAxis_tasks5_chain() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks5_chain); }
@Test @Timeout(240) public void B_scaleAxis_tasks10_chain() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks10_chain); }
@Test @Timeout(360) public void B_scaleAxis_tasks20_chain() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks20_chain); }
@Test @Timeout(600) public void B_scaleAxis_tasks40_chain() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks40_chain); }
@Test @Timeout(900) public void B_scaleAxis_tasks60_chain() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks60_chain); }
@Test @Timeout(180) public void B_scaleAxis_tasks5_fanout() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks5_fanout); }
@Test @Timeout(300) public void B_scaleAxis_tasks10_fanout() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks10_fanout); }
@Test @Timeout(600) public void B_scaleAxis_tasks20_fanout() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks20_fanout); }
@Test @Timeout(1200) public void B_scaleAxis_tasks40_fanout() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks40_fanout); }
@Test @Timeout(2400) public void B_scaleAxis_tasks60_fanout() { assertResultsMatchSolverLN(
    EvaluationSuite::B_scaleAxis_tasks60_fanout); }

// =====
// Partition C1: Fan-out caller topologies
// =====

@Test @Timeout(240) public void C1_fanOut2_caller_c1() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut2_caller_c1); }
@Test @Timeout(240) public void C1_fanOut2_caller_c2() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut2_caller_c2); }
@Test @Timeout(240) public void C1_fanOut3_caller_c1() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut3_caller_c1); }
@Test @Timeout(240) public void C1_fanOut3_caller_c2() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut3_caller_c2); }
@Test @Timeout(240) public void C1_fanOut4_caller_c1() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut4_caller_c1); }
@Test @Timeout(240) public void C1_fanOut4_caller_c2() { assertResultsMatchSolverLN(
    EvaluationSuite::C1_fanOut4_caller_c2); }

// =====
// Partition C2: INF-scheduled hosted tasks
// =====

@Test @Timeout(180) public void C2_infTaskSimple() { assertResultsMatchSolverLN(
    EvaluationSuite::C2_infTaskSimple); }
@Test @Timeout(180) public void C2_infTaskShared() { assertResultsMatchSolverLN(
    EvaluationSuite::C2_infTaskShared); }
@Test @Timeout(240) public void C2_infTaskDeep() { assertResultsMatchSolverLN(
    EvaluationSuite::C2_infTaskDeep); }

// =====
// Partition C3: Multi-class + multi-entry interaction
// =====

@Test @Timeout(180) public void C3_multiclassMultiEntry_2refs() {
    assertResultsMatchSolverLN(EvaluationSuite::C3_multiclassMultiEntry_2refs); }
@Test @Timeout(180) public void C3_multiclassMultiEntry_3refs() {
    assertResultsMatchSolverLN(EvaluationSuite::C3_multiclassMultiEntry_3refs); }
@Test @Timeout(180) public void C3_multiclassMultiEntry_asymmetric() {
    assertResultsMatchSolverLN(EvaluationSuite::C3_multiclassMultiEntry_asymmetric); }

```

```

// =====
// Partition C4: Boundary cases for ablation
// =====

@Test @Timeout(240) public void C4_nearSaturation()    { assertResultsMatchSolverLN(
    EvaluationSuite::C4_nearSaturation);    }
@Test @Timeout(240) public void C4_predicateThreshold() { assertResultsMatchSolverLN(
    EvaluationSuite::C4_predicateThreshold); }
@Test @Timeout(240) public void C4_lowDemand()         { assertResultsMatchSolverLN(
    EvaluationSuite::C4_lowDemand);         }

// =====
// Partition C5: Saturation regime (boundary -- expected divergence)
// =====

@Test @Timeout(300) public void C5_satFanout_tasks5()    { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanout_tasks5);    }
@Test @Timeout(600) public void C5_satFanout_tasks10()   { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanout_tasks10);   }
@Test @Timeout(900) public void C5_satFanout_tasks20()   { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanout_tasks20);   }
@Test @Timeout(300) public void C5_satFanoutCaller_w2()  { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanoutCaller_w2);  }
@Test @Timeout(300) public void C5_satFanoutCaller_w3()  { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanoutCaller_w3);  }
@Test @Timeout(300) public void C5_satFanoutCaller_w4()  { assertResultsMatchSolverLN(
    EvaluationSuite::C5_satFanoutCaller_w4);  }

// =====
// Partition C6: AND-fork join-correction boundary
// =====

@Test @Timeout(180) public void C6_andForkPsHost()      { assertResultsMatchSolverLN(
    EvaluationSuite::C6_andForkPsHost);      }
@Test @Timeout(180) public void C6_multiclassAndFork()  { assertResultsMatchSolverLN(
    EvaluationSuite::C6_multiclassAndFork);  }

// =====
// Partition C7: Deep-chain stress (convergence-rate boundary)
// =====

@Test @Timeout(240) public void C7_deepChainMultiCall() { assertResultsMatchSolverLN(
    EvaluationSuite::C7_deepChainMultiCall); }
@Test @Timeout(240) public void C7_deepChainAllExp()   { assertResultsMatchSolverLN(
    EvaluationSuite::C7_deepChainAllExp);   }

```